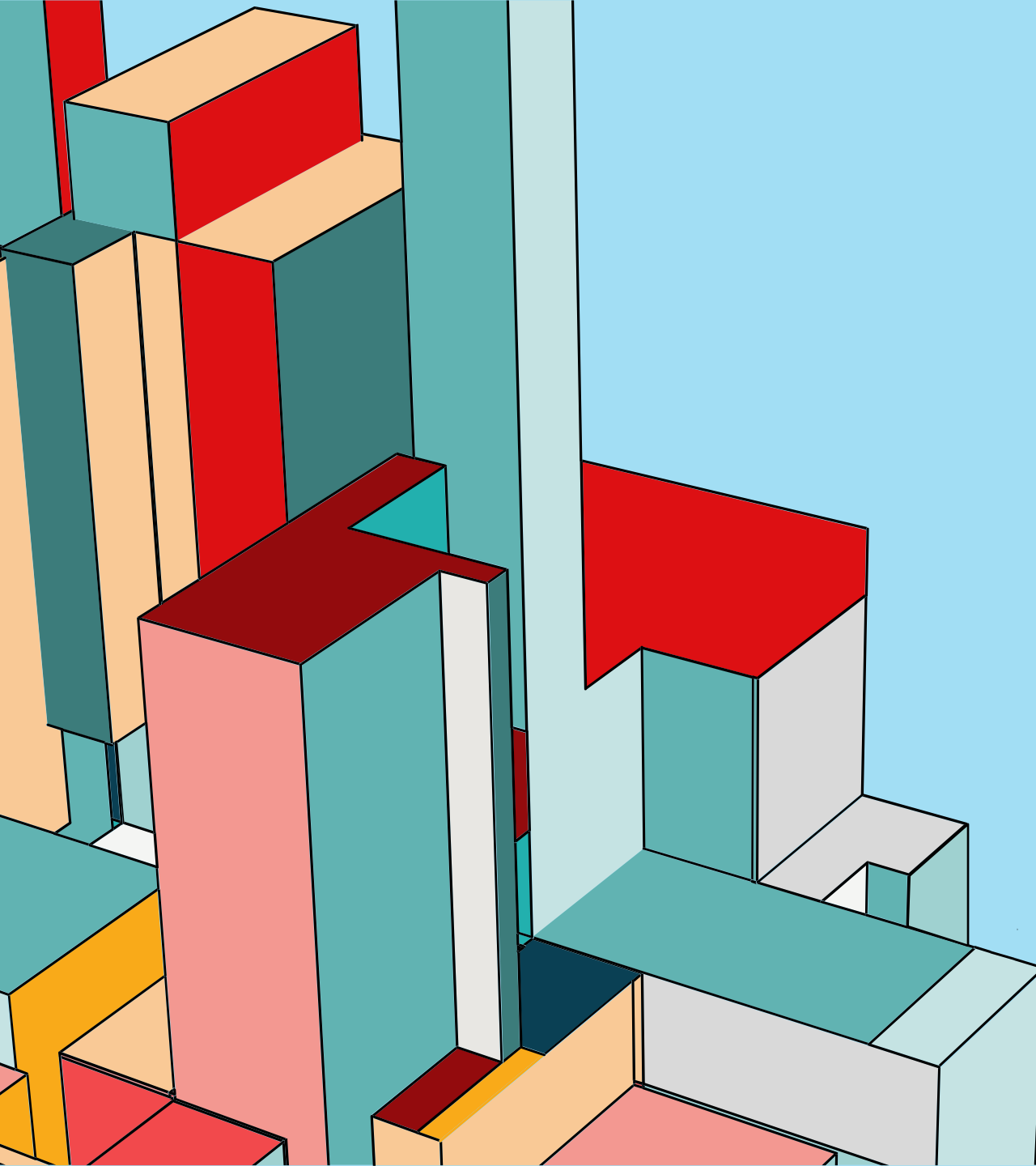


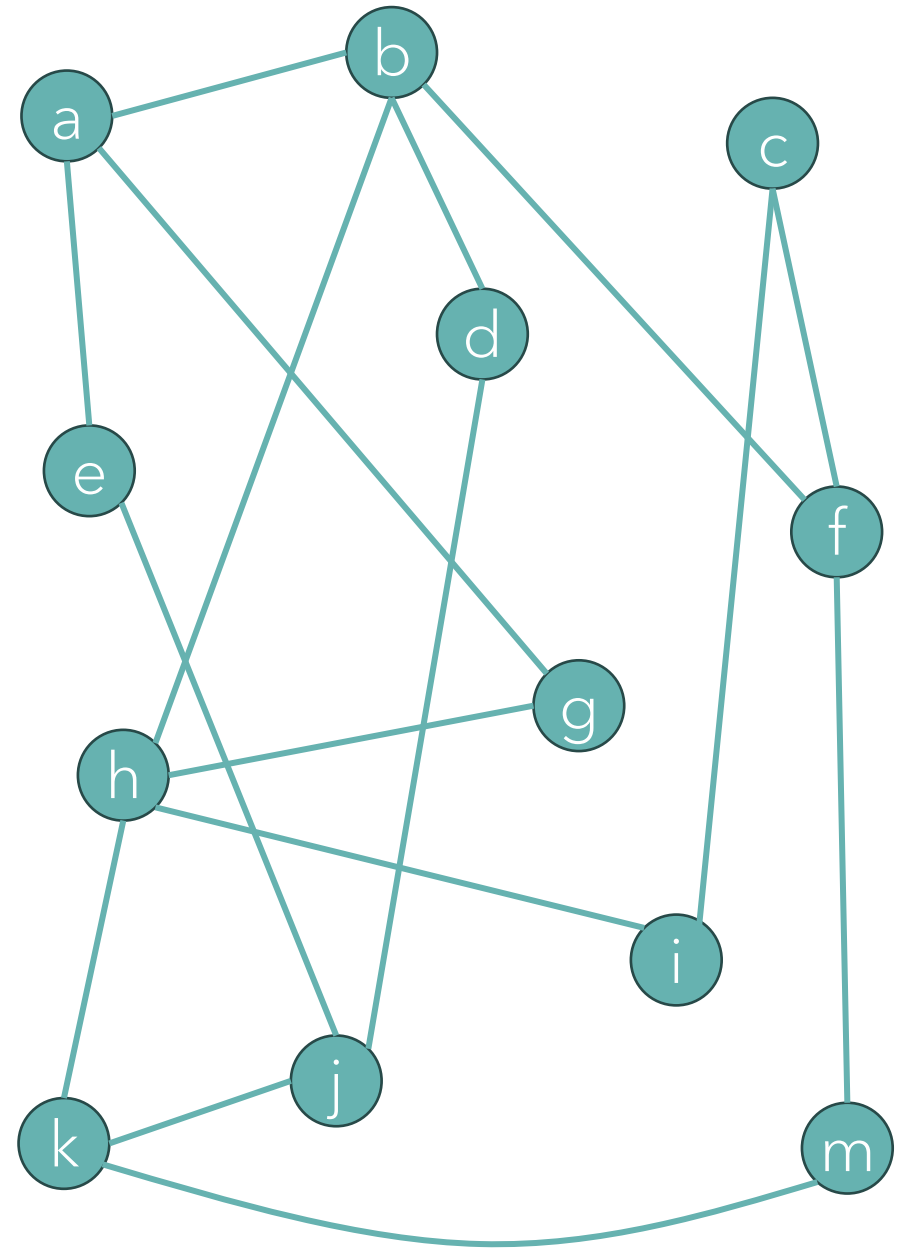


LECTURE 5
SUPPLEMENT
BFS, DFS, AND
TOPO SORT
DEMONSTRATION

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

REVIEW OF BFS / DFS

We'll take a few minutes here to walk through a full execution of each search algorithm.

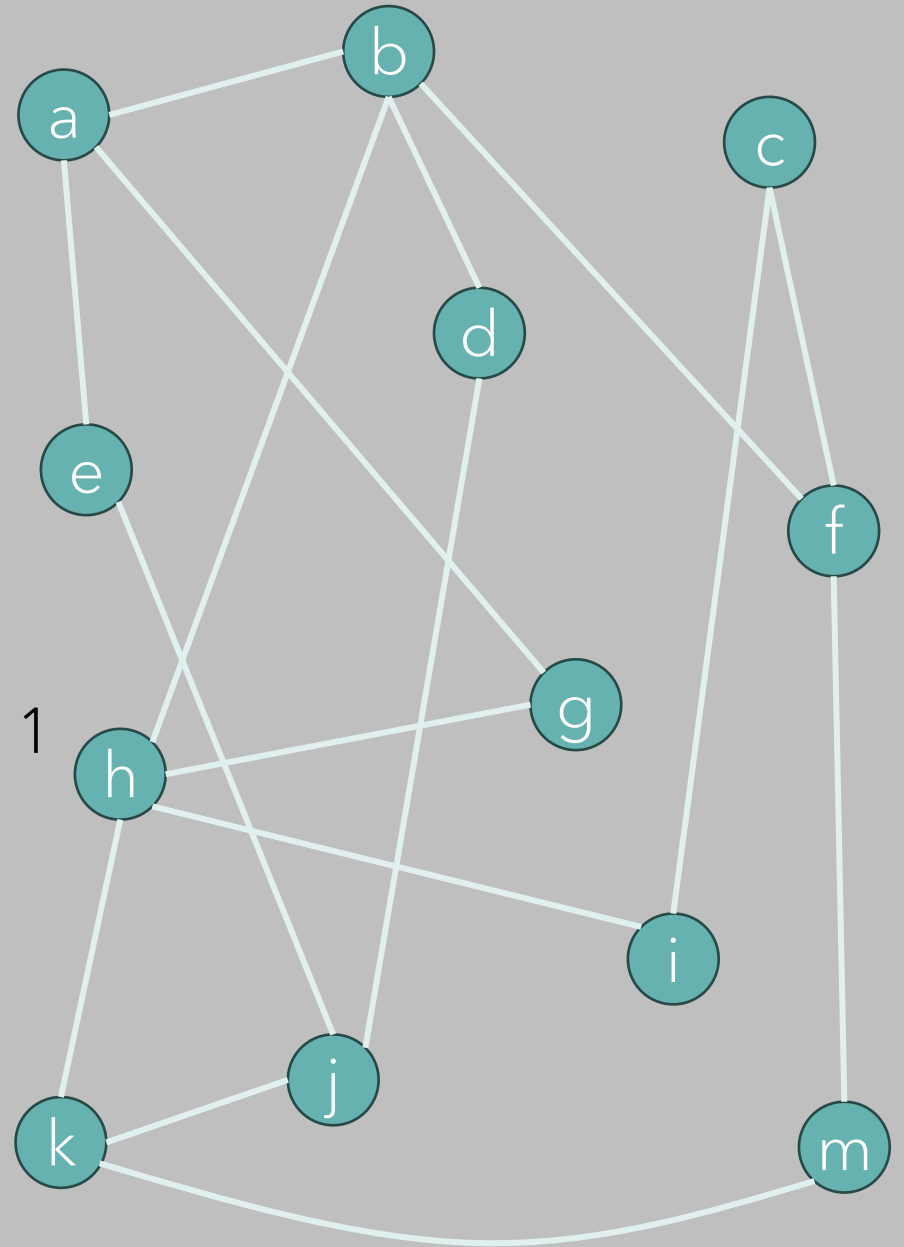


BREADTH-FIRST SEARCH

We'll start at h this time.

h goes into the queue and gets label 1.

Queue: h

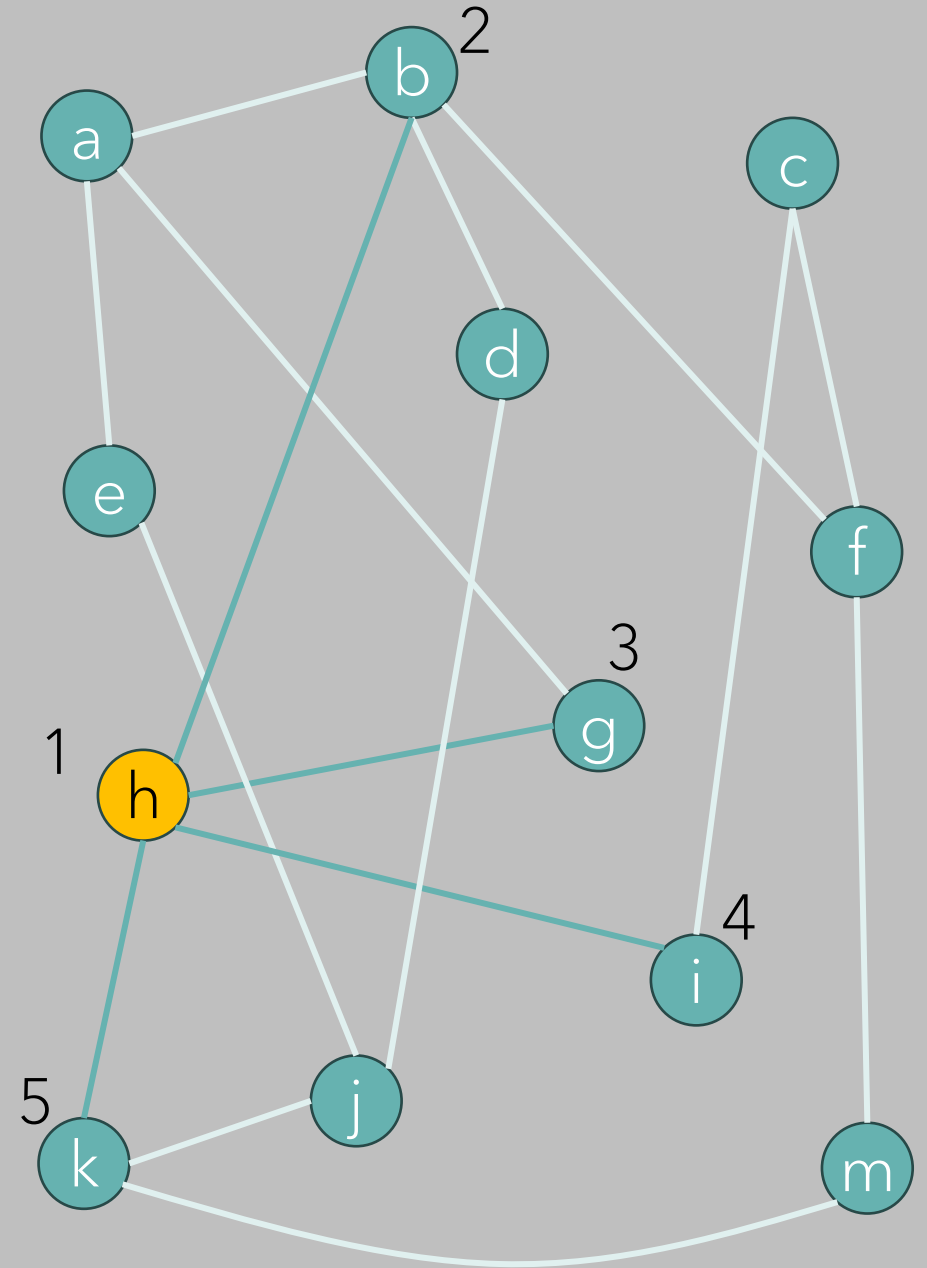


BREADTH-FIRST SEARCH

We take h out of the queue and process it.

Each of h 's unlabeled neighbors goes into the queue with a new label.

Queue: b, g, i, k

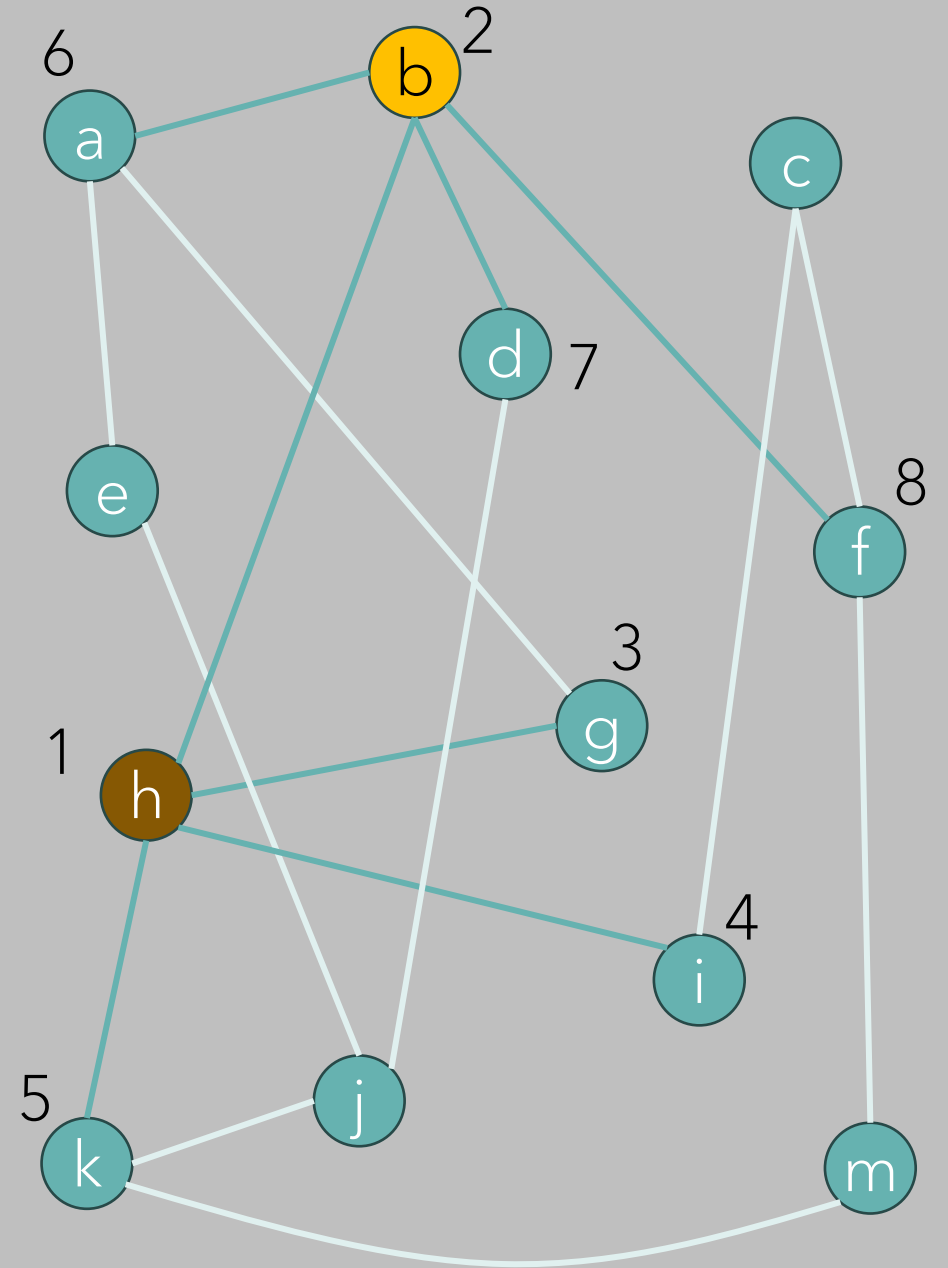


BREADTH-FIRST SEARCH

We are now done with *h*. *b* is next in the queue.

Each of *b*'s unlabeled neighbors goes into the queue with a new label.

Queue: *g, i, k, a, d, f*



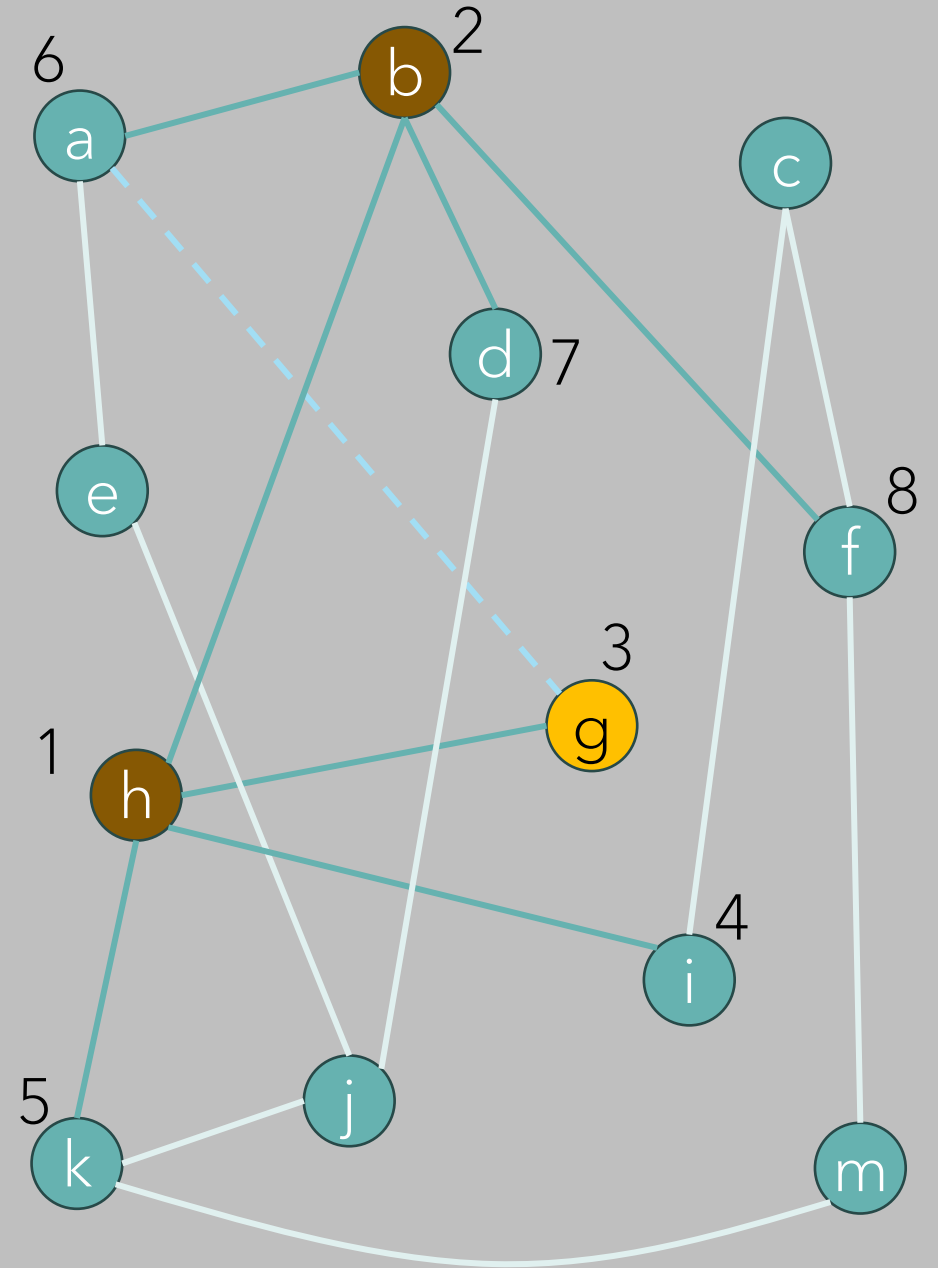
BREADTH-FIRST SEARCH

We process g next.

Each of g 's unlabeled neighbors goes into the queue – but there aren't any.

g has a labeled neighbor at a ; this will become a cross edge.

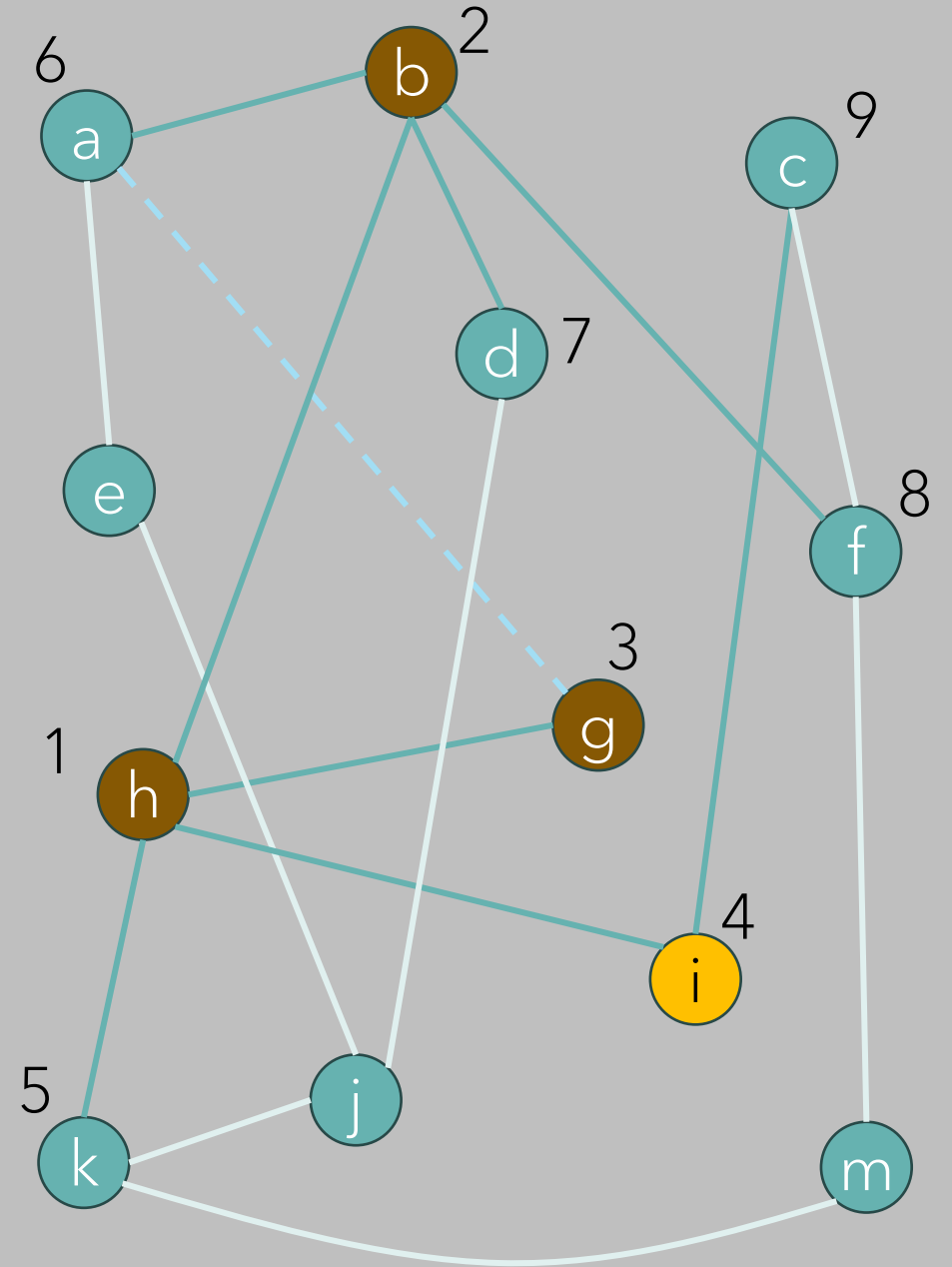
Queue: i, k, a, d, f



BREADTH-FIRST SEARCH

We process i next, adding c to the queue.

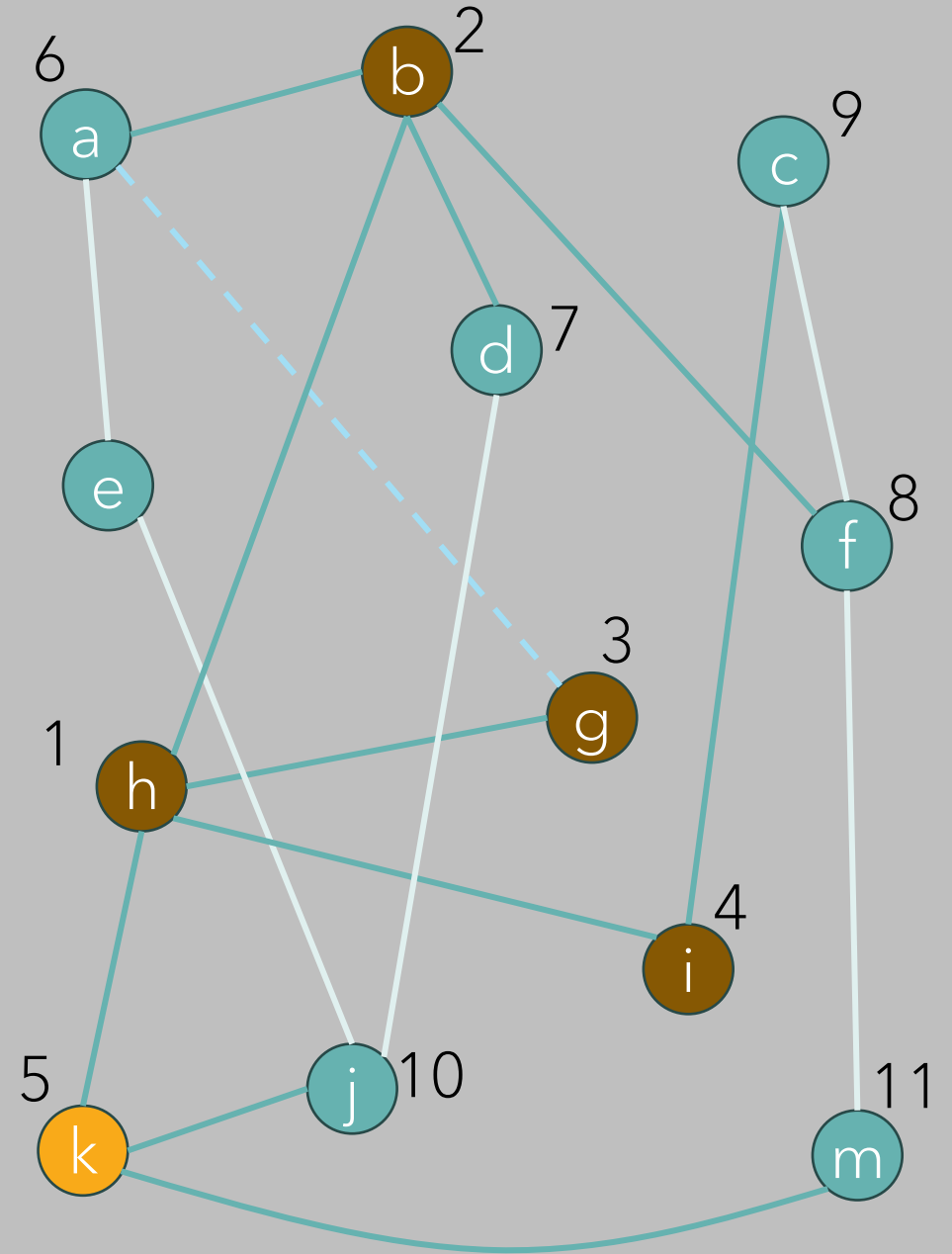
Queue: k, a, d, f, c



BREADTH-FIRST SEARCH

We process k next, adding both j and m to the queue.

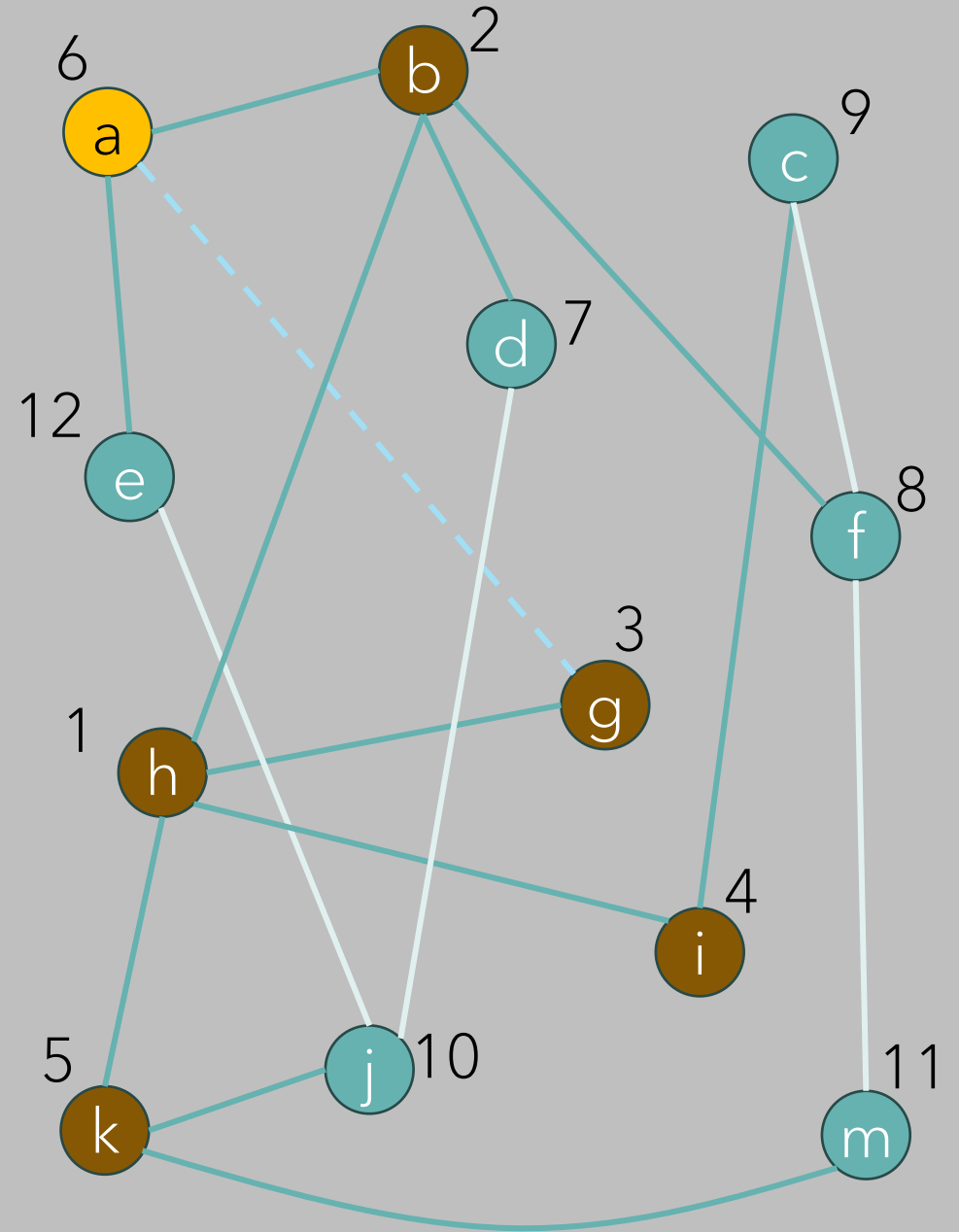
Queue: a, d, f, c, j, m



BREADTH-FIRST SEARCH

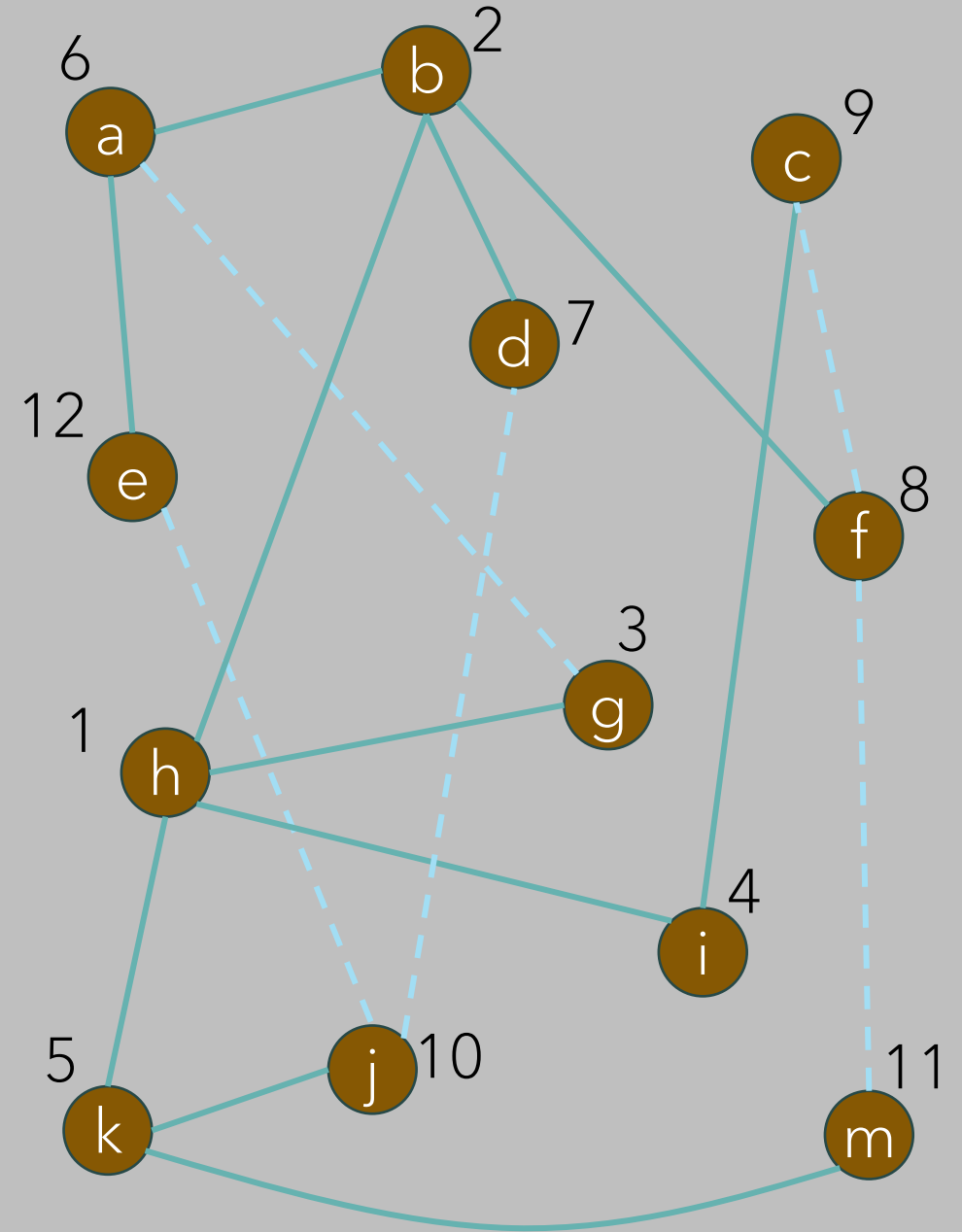
We process *a* next, adding *e* to the queue.

Queue: *d, f, c, j, m, e*



BREADTH-FIRST SEARCH

At this point, there are no more unlabeled nodes. Each node will be removed from the queue in order; all remaining edges become cross edges.

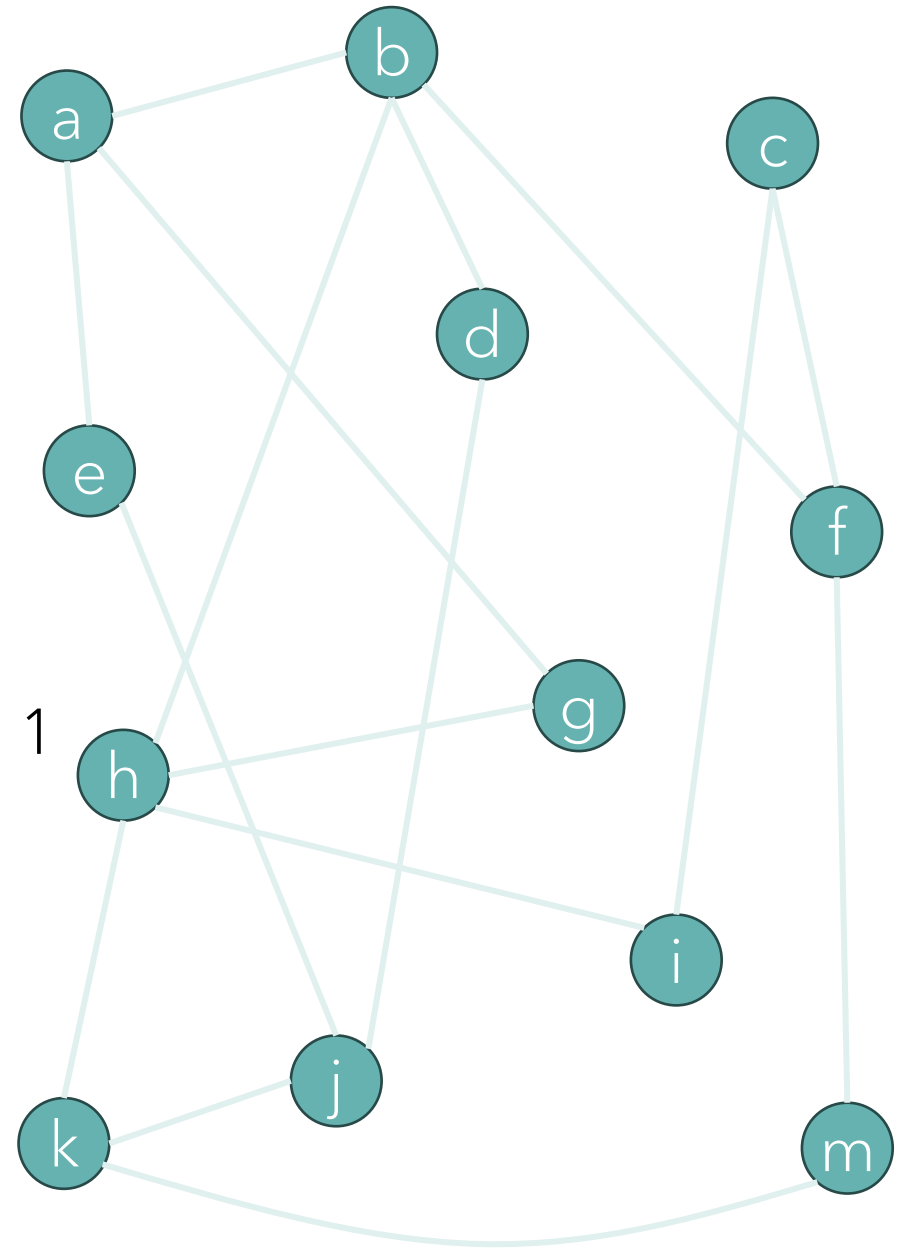


DEPTH-FIRST SEARCH

Once again starting at h .

h goes onto the stack with an "in" time of 1.

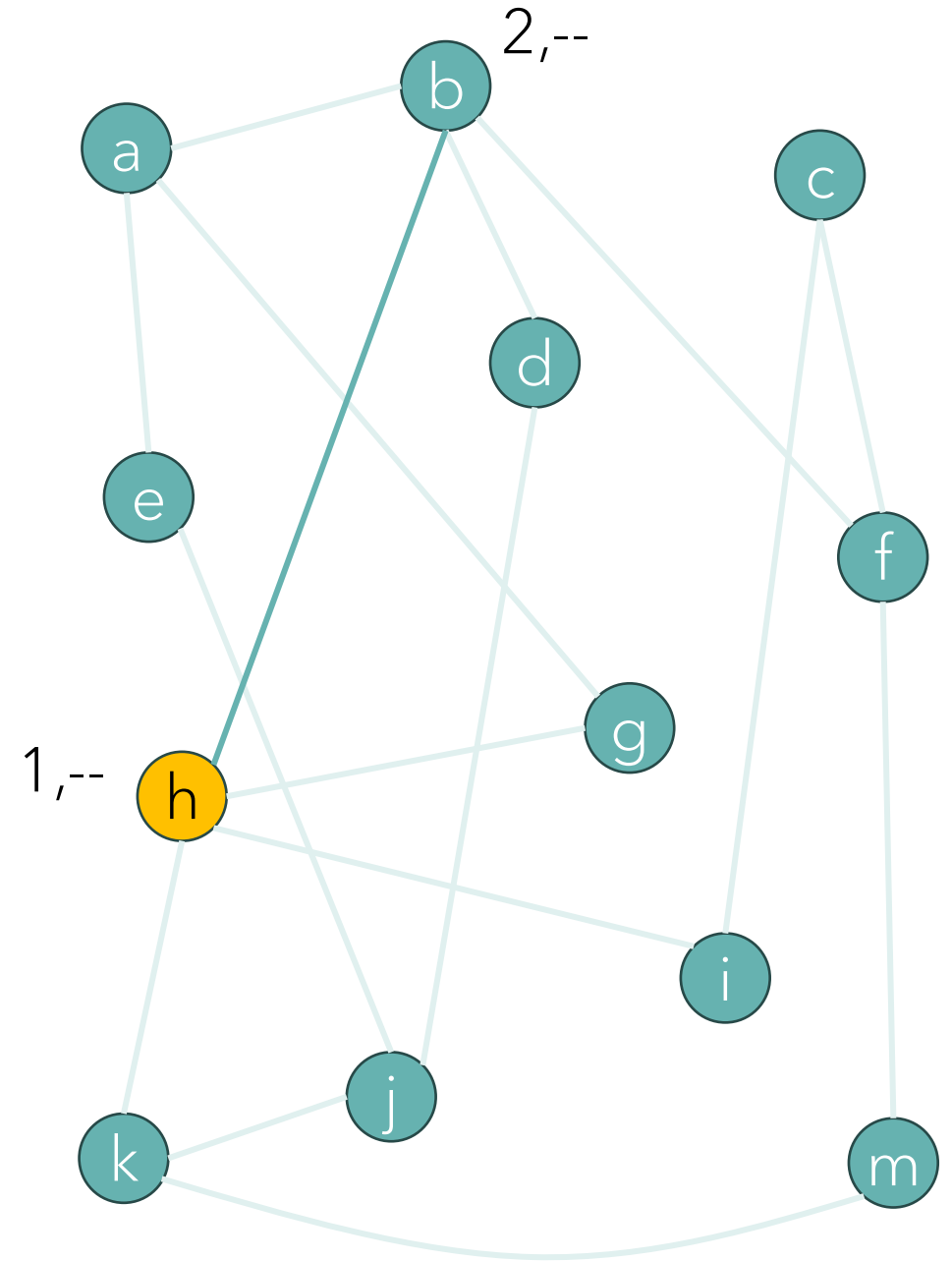
Stack: h



DEPTH-FIRST SEARCH

We add h 's first neighbor to the stack (creating a tree edge). b gets an "in" time of 2.

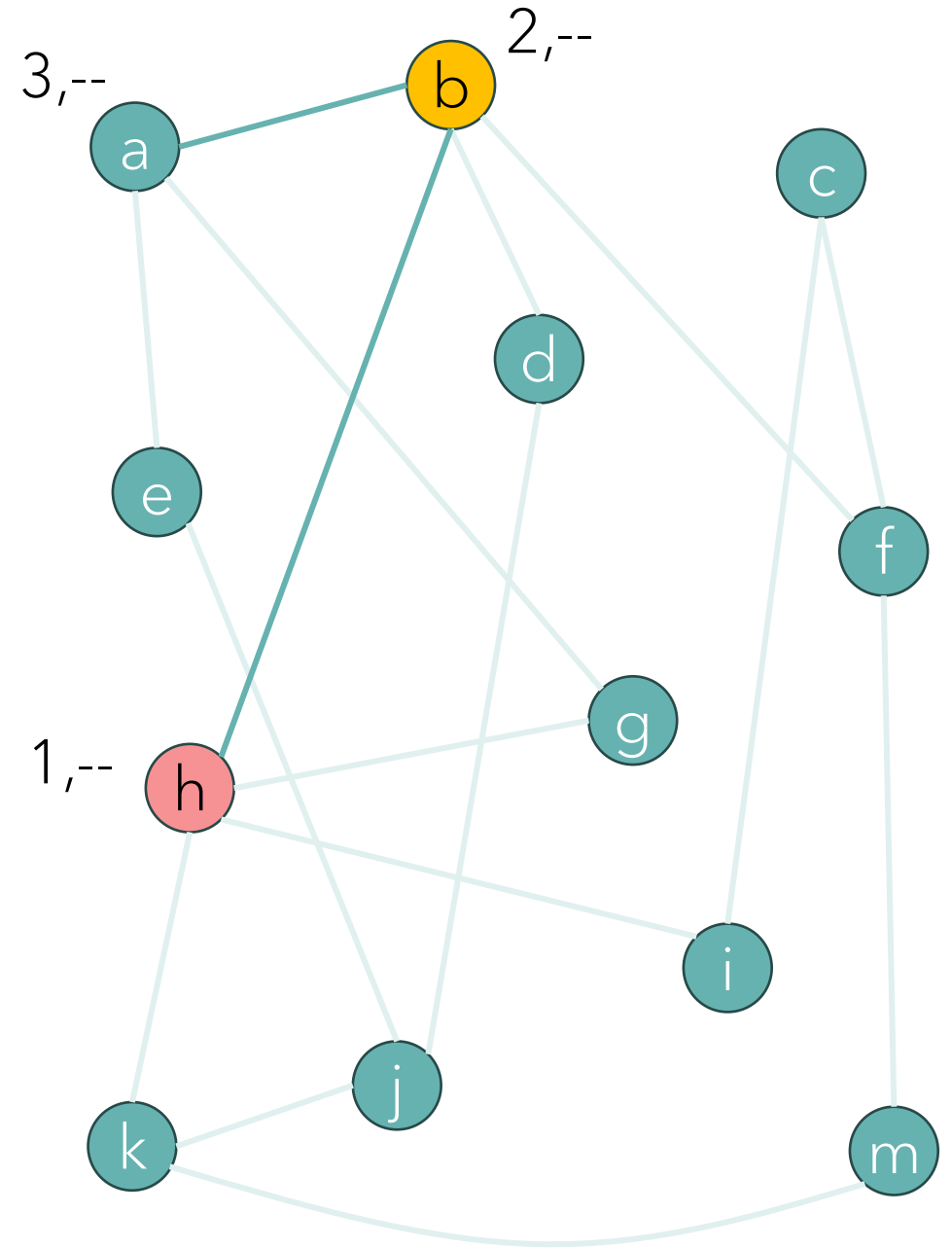
Stack: b, h



DEPTH-FIRST SEARCH

b is at the top of the stack. We push its first unlabeled neighbor onto the stack.

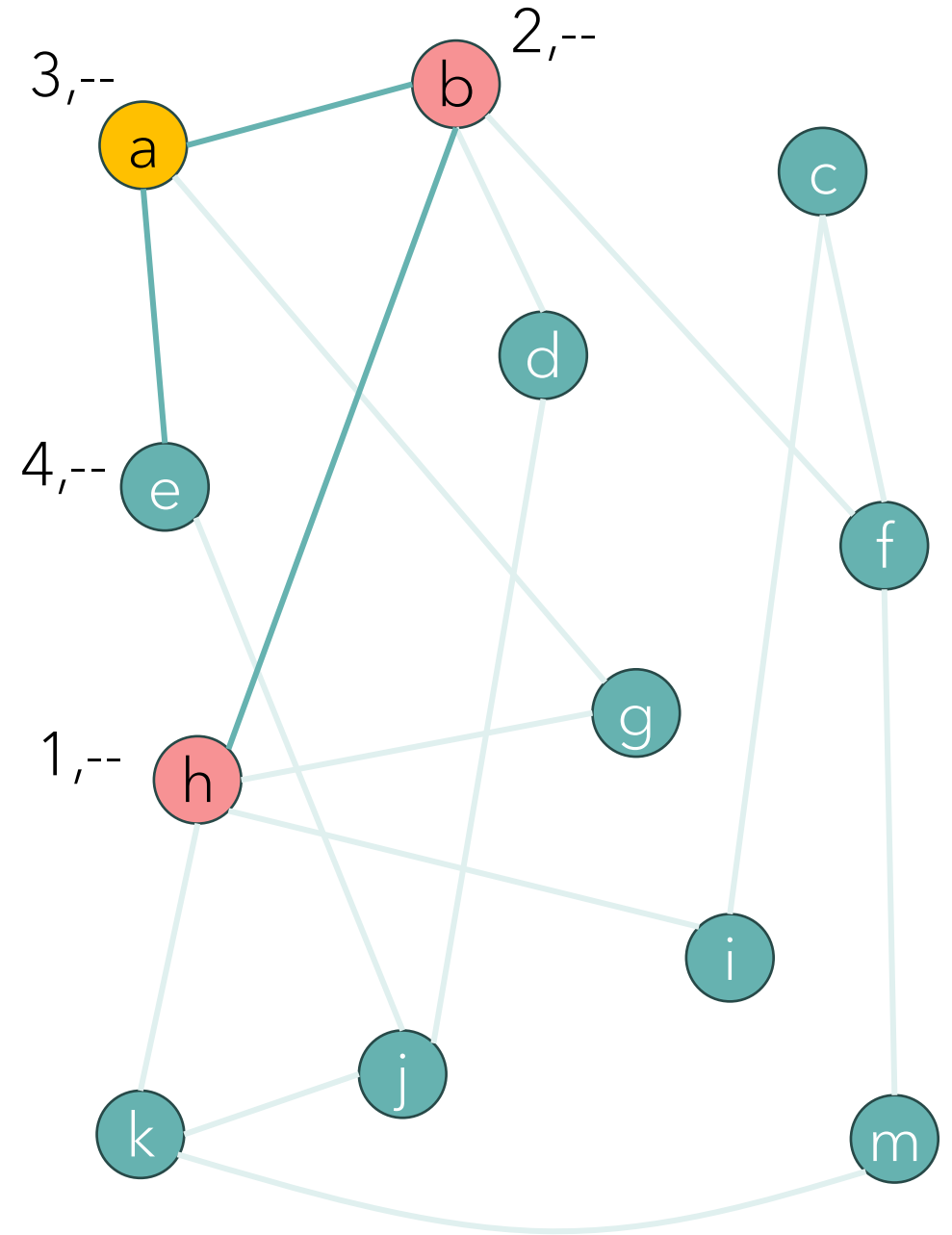
Stack: *a, b, h*



DEPTH-FIRST SEARCH

a is at the top of the stack. We push its first unlabeled neighbor onto the stack.

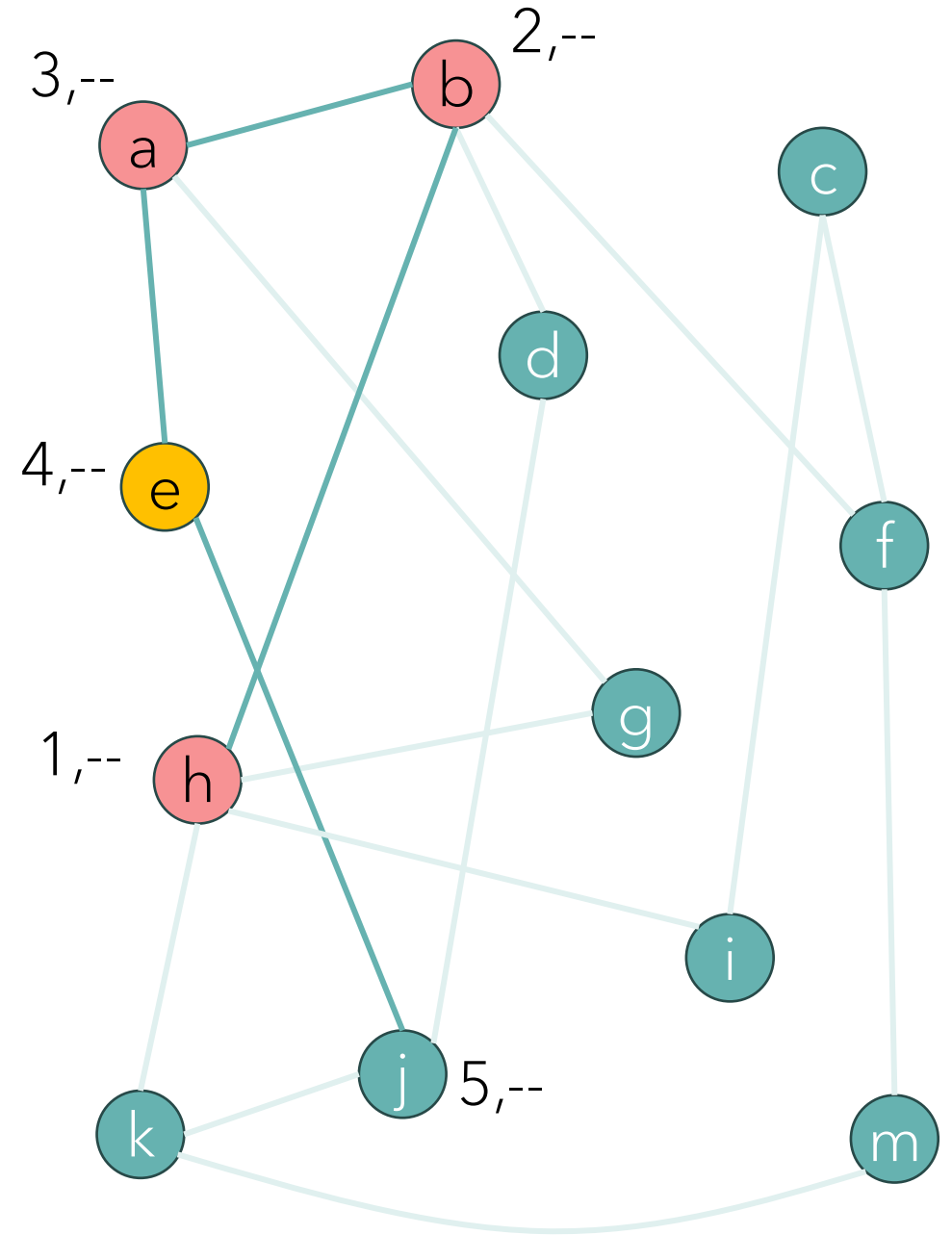
Stack: *e, a, b, h*



DEPTH-FIRST SEARCH

e is at the top of the stack. We push its first unlabeled neighbor onto the stack.

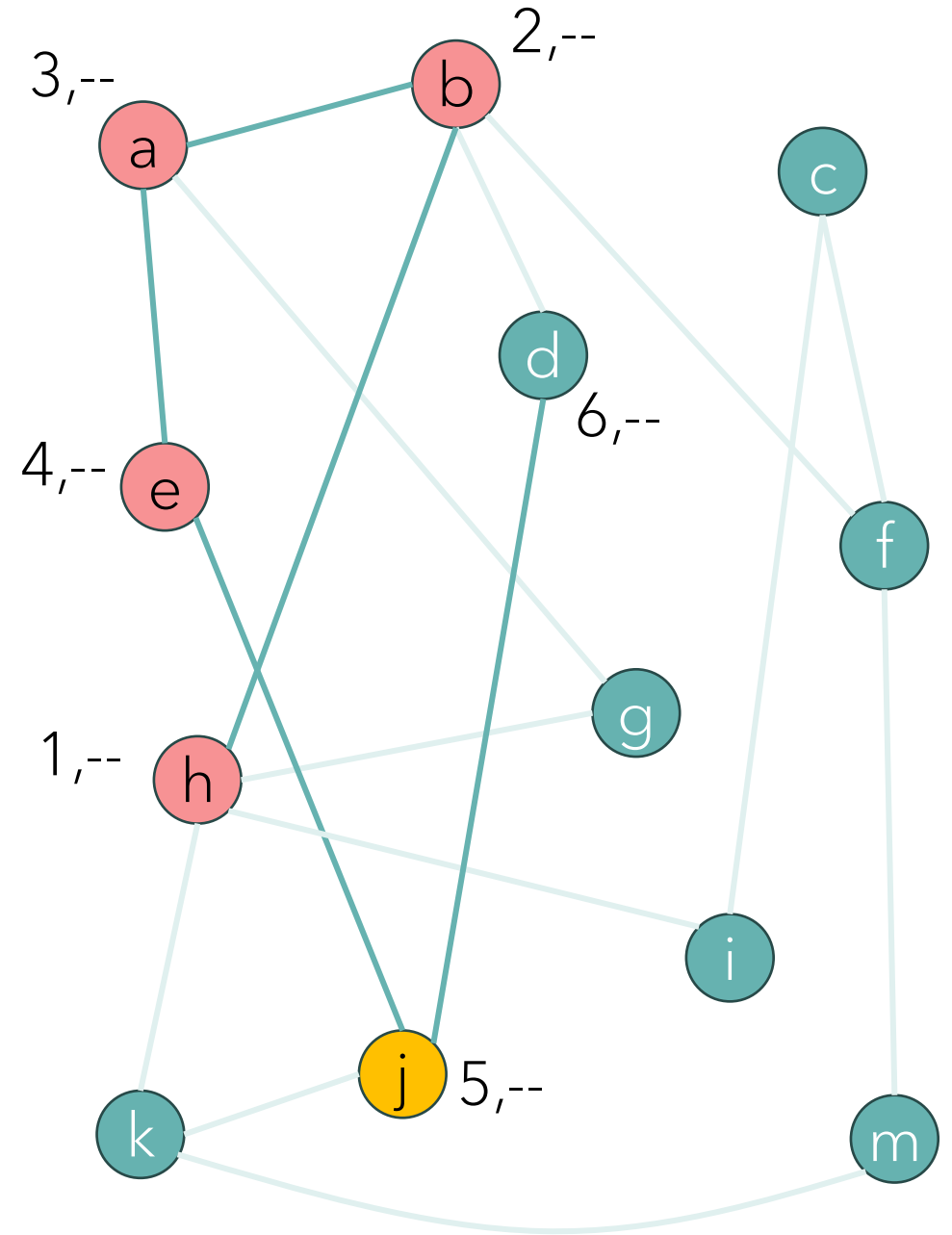
Stack: j, e, a, b, h



DEPTH-FIRST SEARCH

j is at the top of the stack. We push its first unlabeled neighbor onto the stack.

Stack: d, j, e, a, b, h

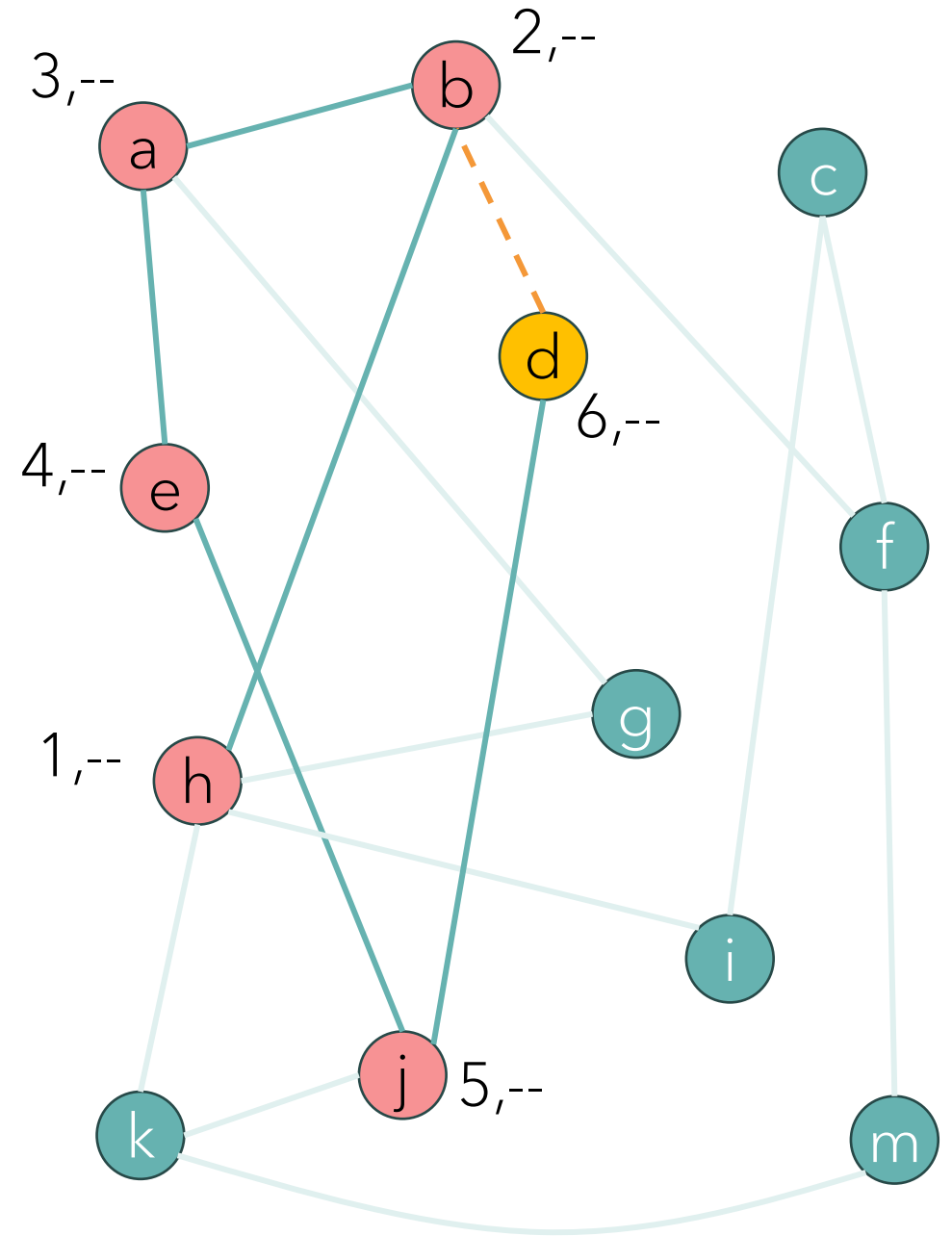


DEPTH-FIRST SEARCH

d is at the top of the stack.

Its first neighbor already has an "in" time, so that edge becomes a back edge.

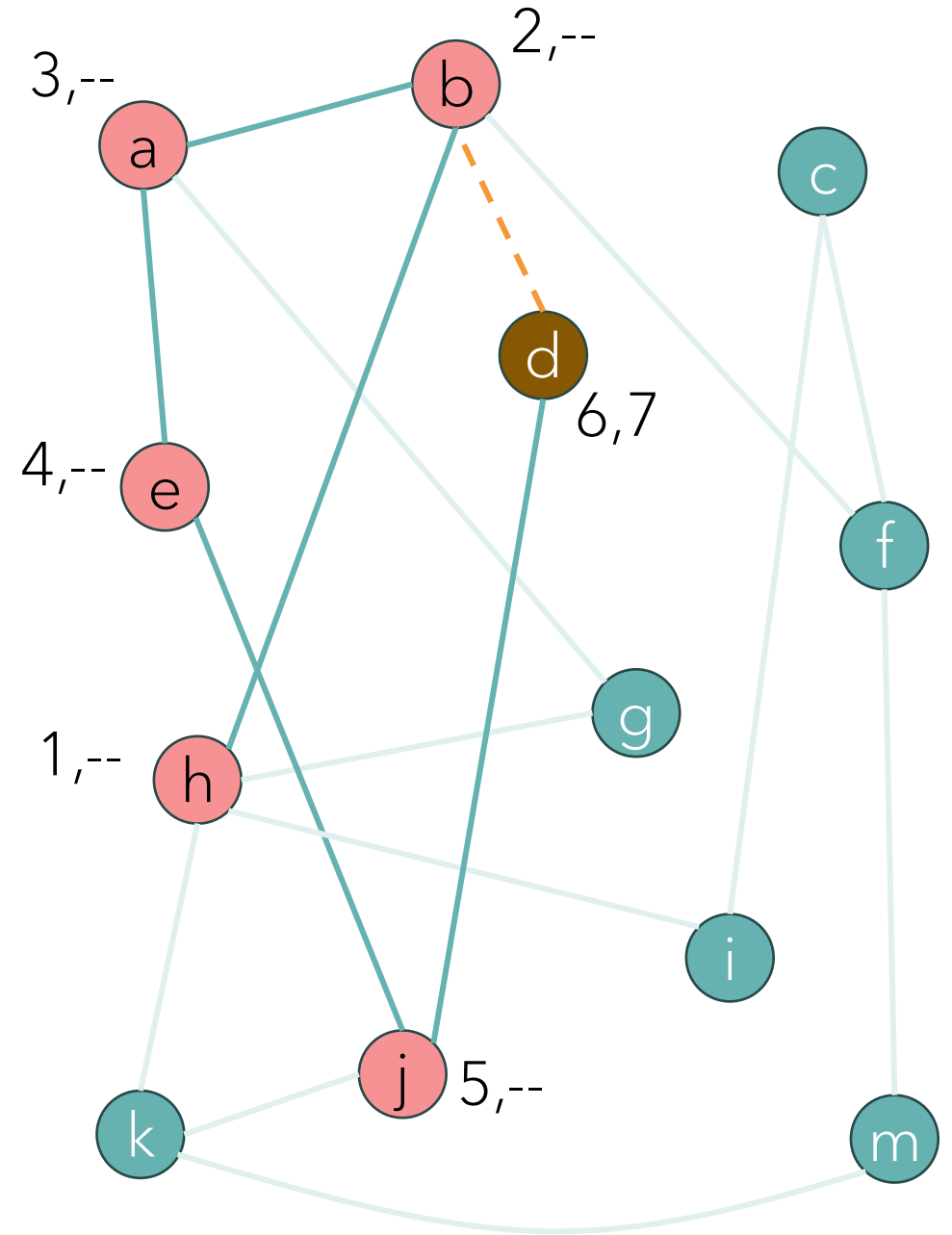
Stack: d, j, e, a, b, h



DEPTH-FIRST SEARCH

d has no more neighbors. We finish processing it by assigning its "out" time and popping it off the stack.

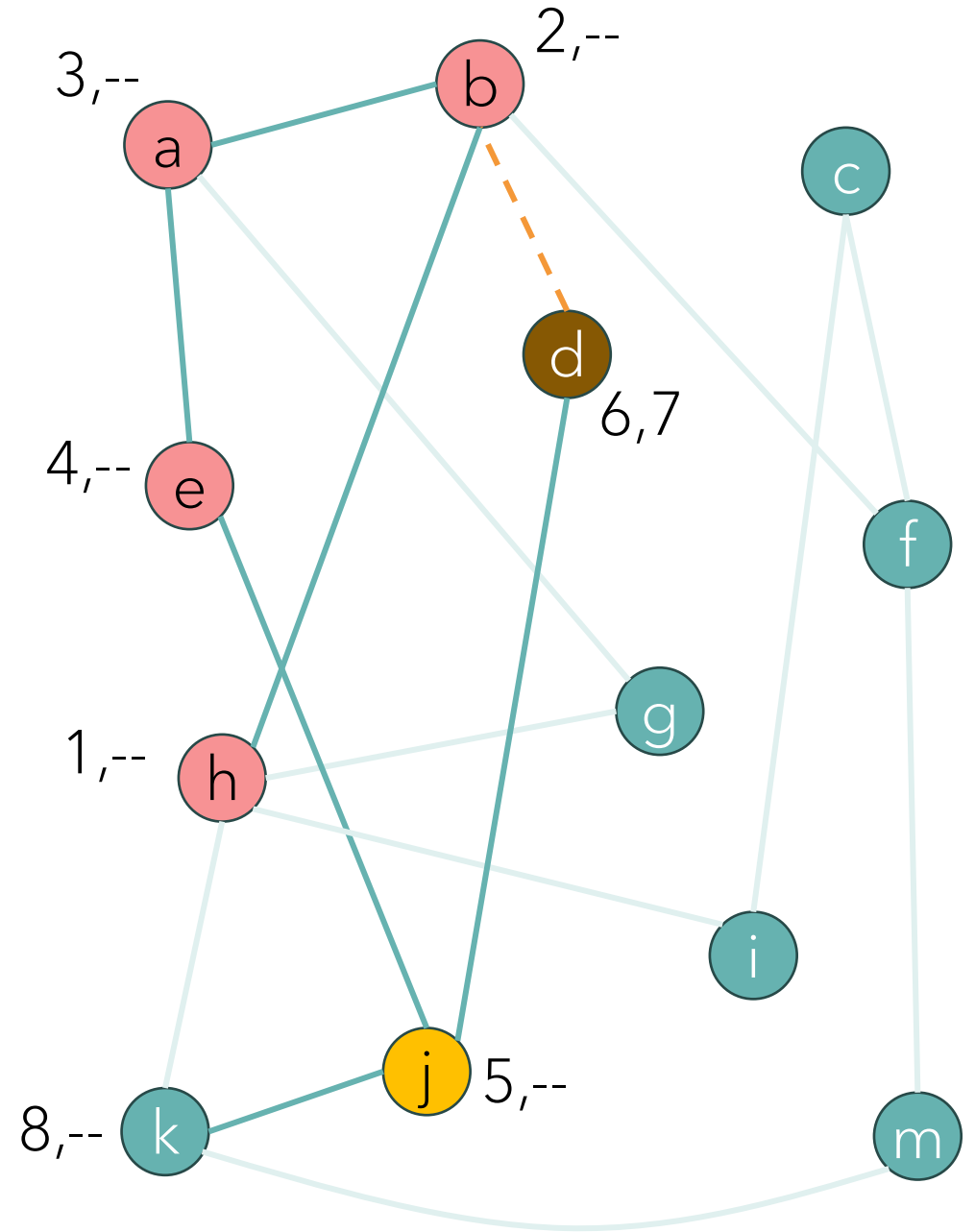
Stack: *j, e, a, b, h*



DEPTH-FIRST SEARCH

j is back on top. Its remaining unlabeled neighbor is *k*, which goes onto the stack.

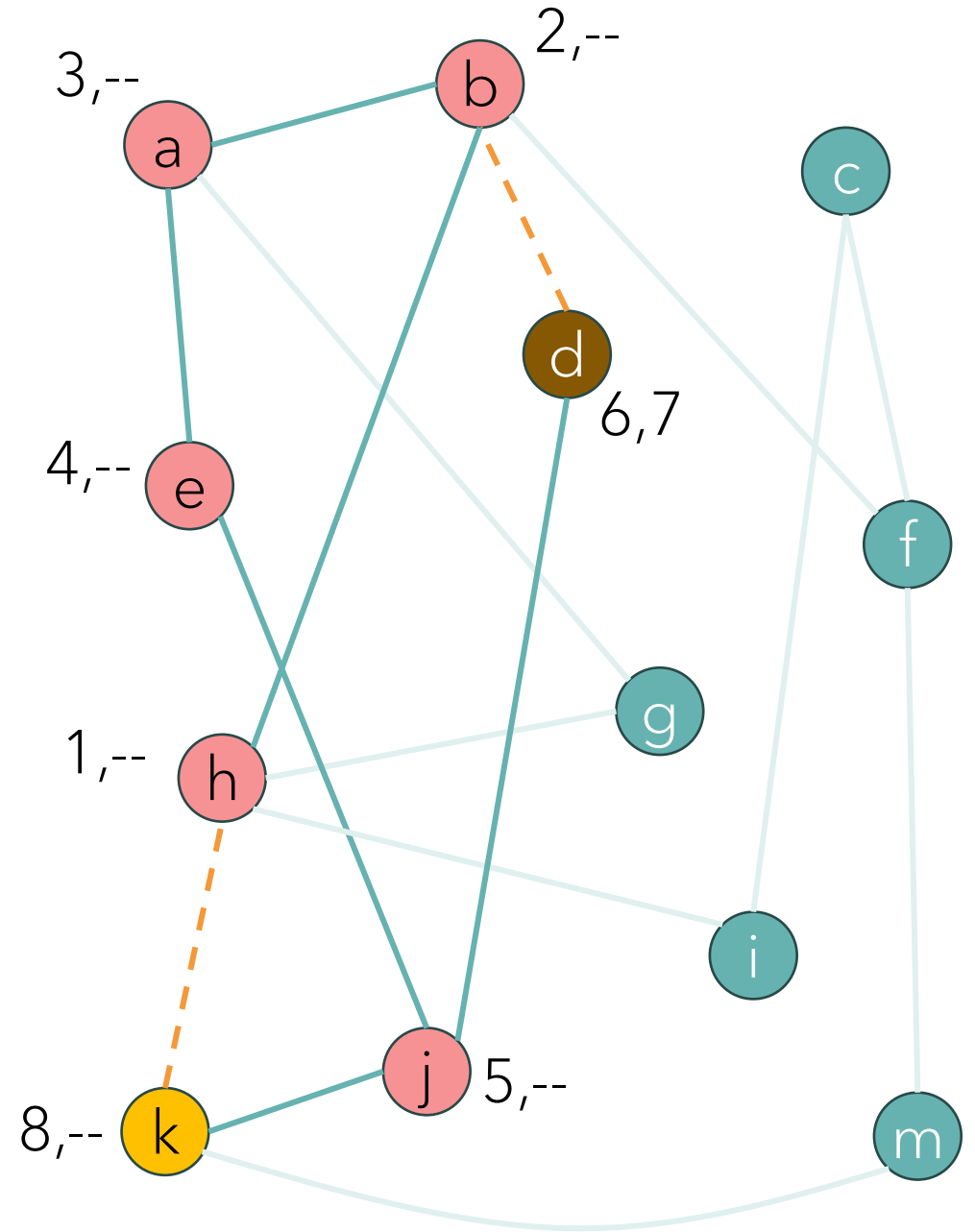
Stack: *k, j, e, a, b, h*



DEPTH-FIRST SEARCH

k 's first edge leads back to h , which has an "in" time, so this is another back edge.

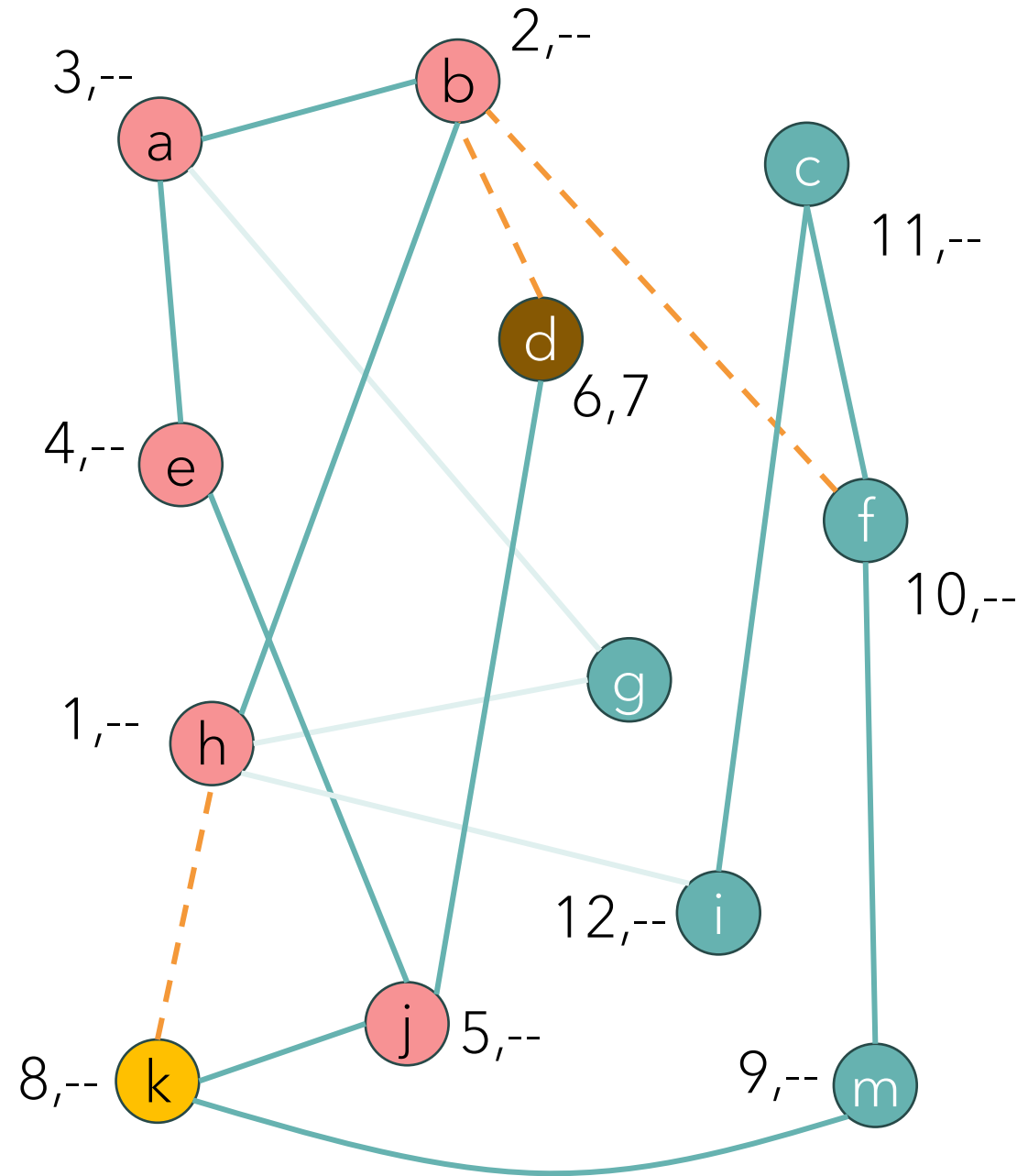
Stack: k, j, e, a, b, h



DEPTH-FIRST SEARCH

Fast forward: In order, m, f, c, i all get pushed onto the stack
(While we are at f , we find another back edge to b before we put c on the stack.)

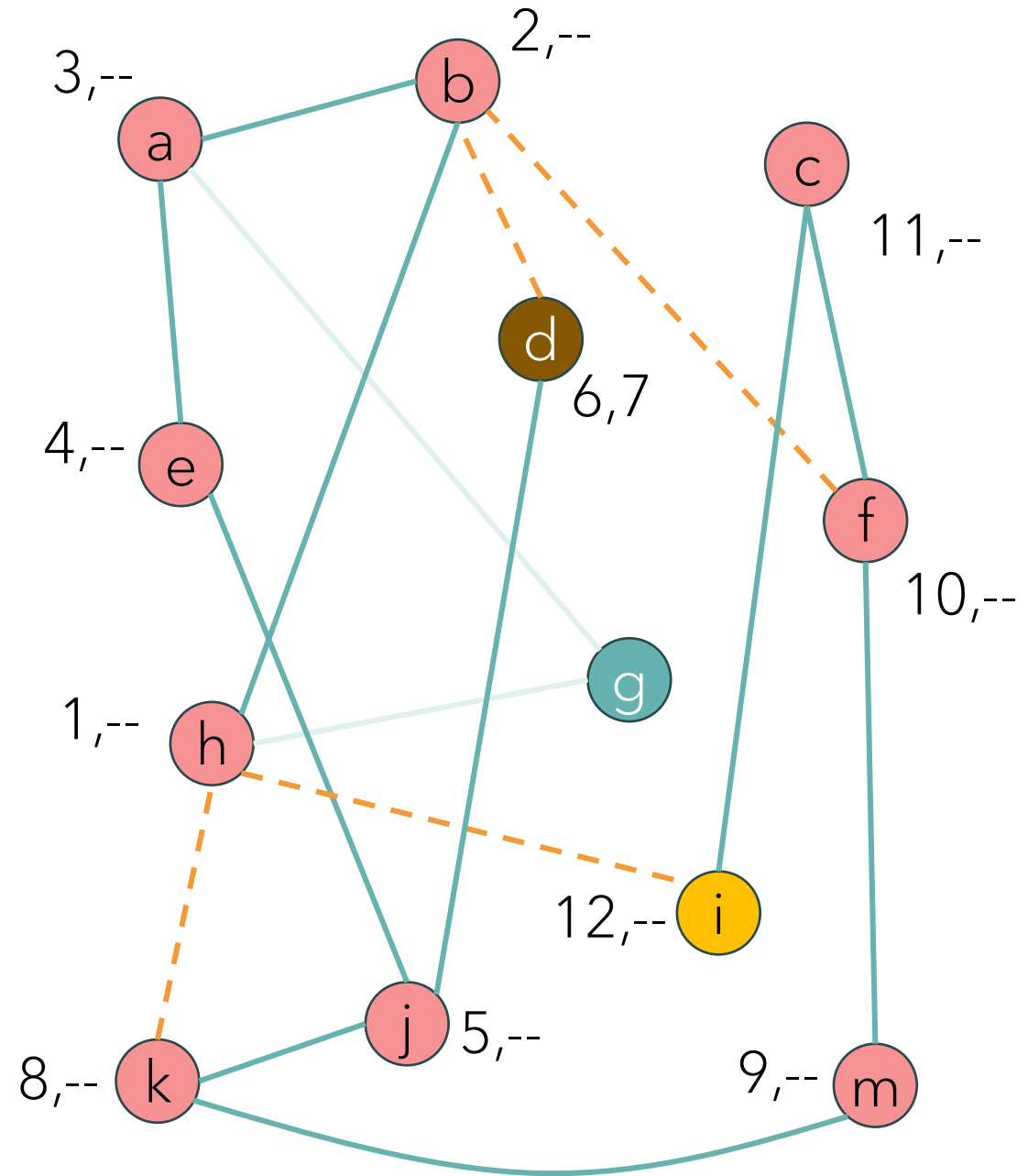
Stack: $i, c, f, m, k, j, e, a, b, h$



DEPTH-FIRST SEARCH

At i we find a back edge to h .

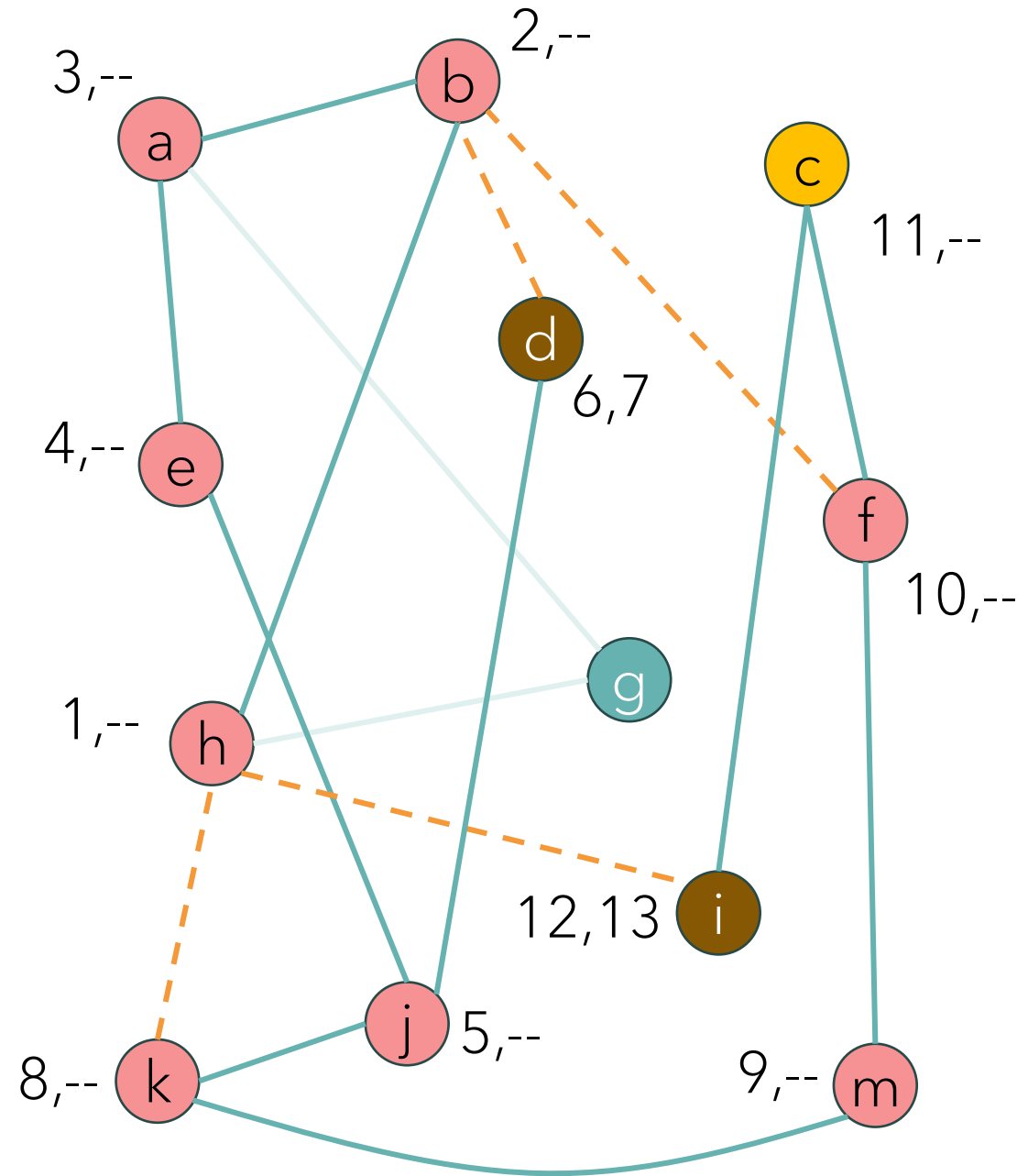
Stack: $i, c, f, m, k, j, e, a, b, h$



DEPTH-FIRST SEARCH

i is now fully explored, so we assign an "out" time and pop the stack.

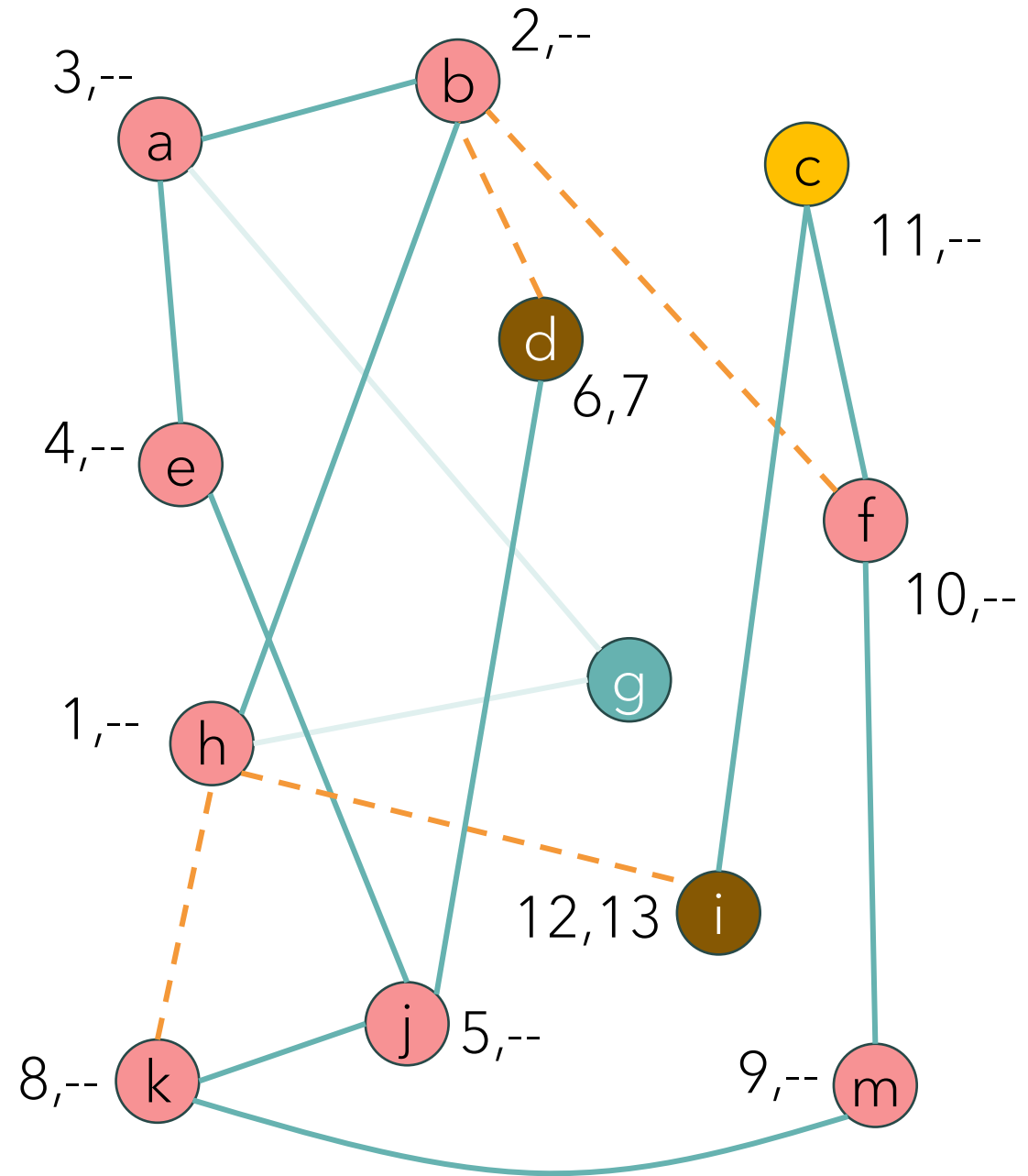
Stack: $c, f, m, k, j, e, a, b, h$



DEPTH-FIRST SEARCH

Fast forward: We have explored all edges of most of the stack. Each item in turn will pop off with an "out" time until we get all the way back down to *a*.

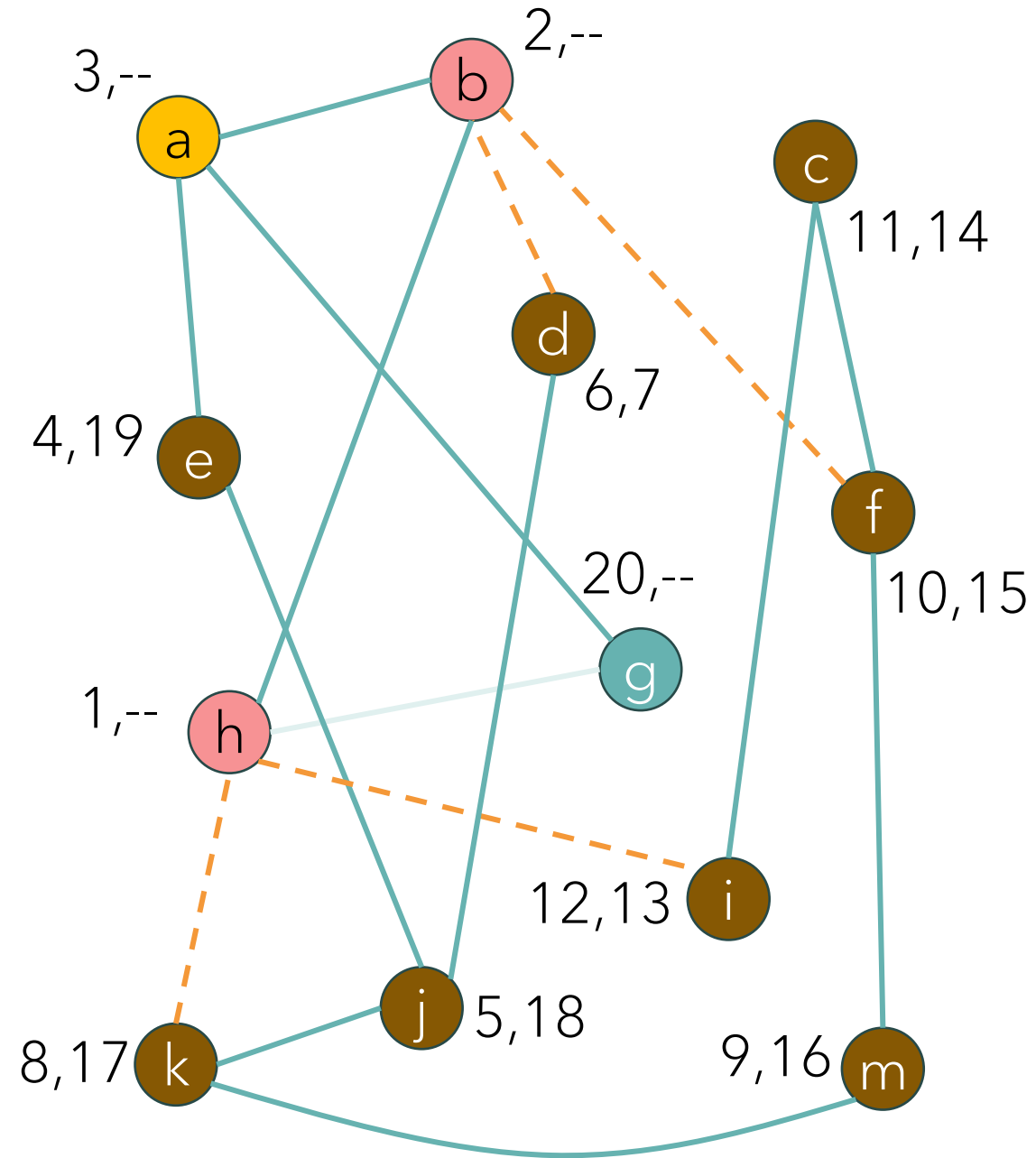
Stack: *c, f, m, k, j, e, a, b, h*



DEPTH-FIRST SEARCH

a still has an unexplored edge to *g*, so *g* gets put onto the stack.

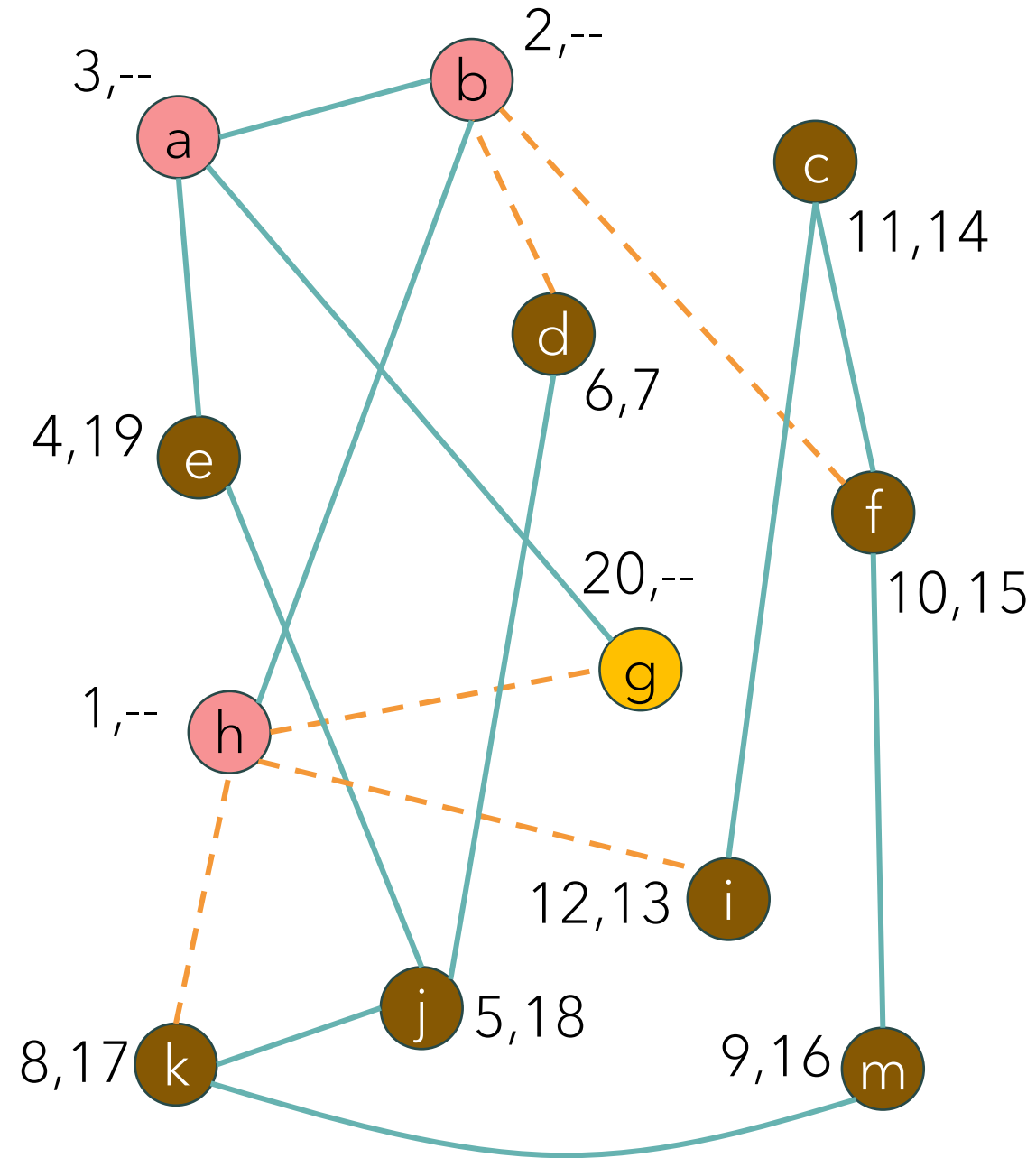
Stack: *g, a, b, h*



DEPTH-FIRST SEARCH

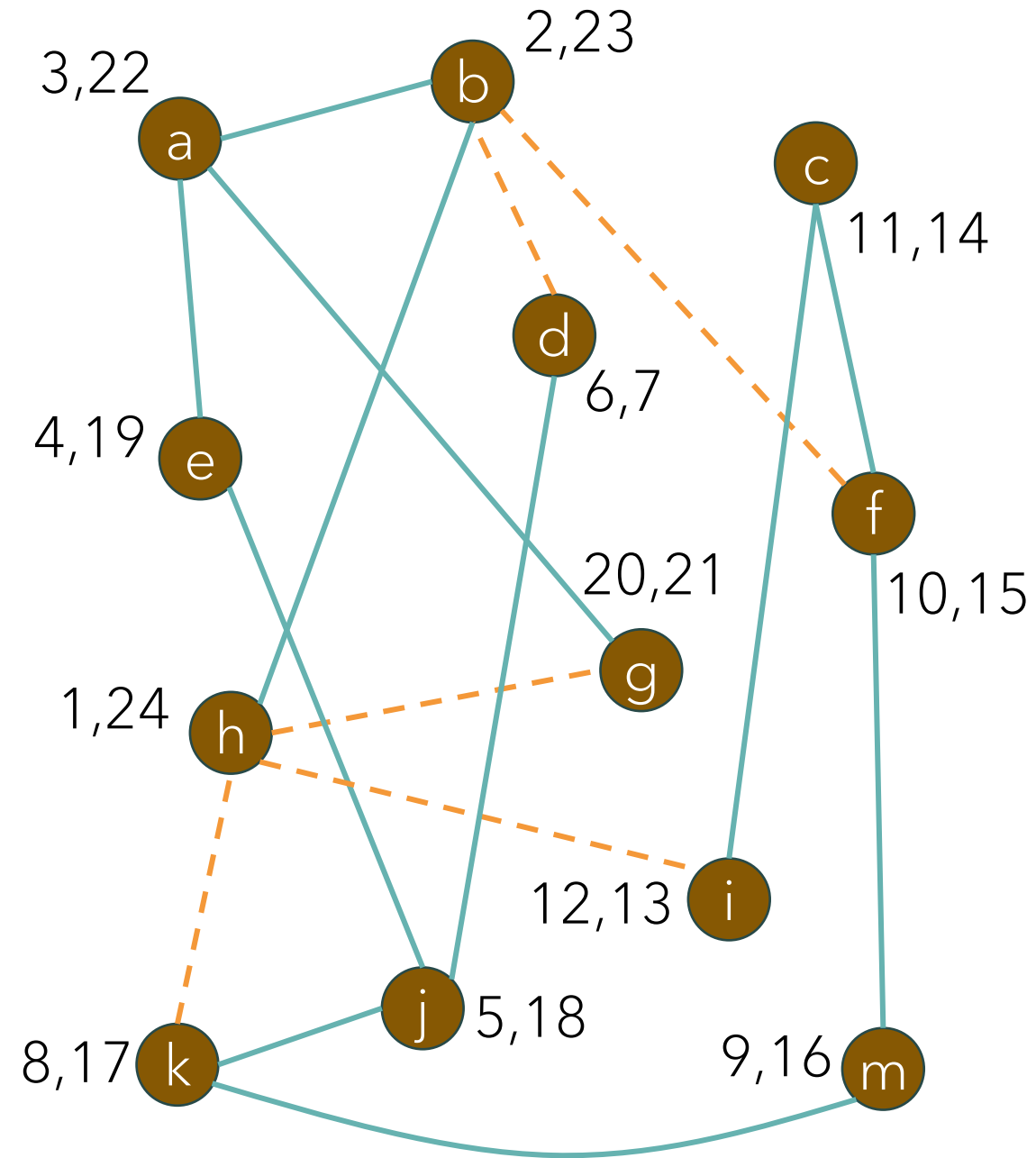
g 's sole unexplored edge is a back edge to h .

Stack: g, a, b, h



DEPTH-FIRST SEARCH

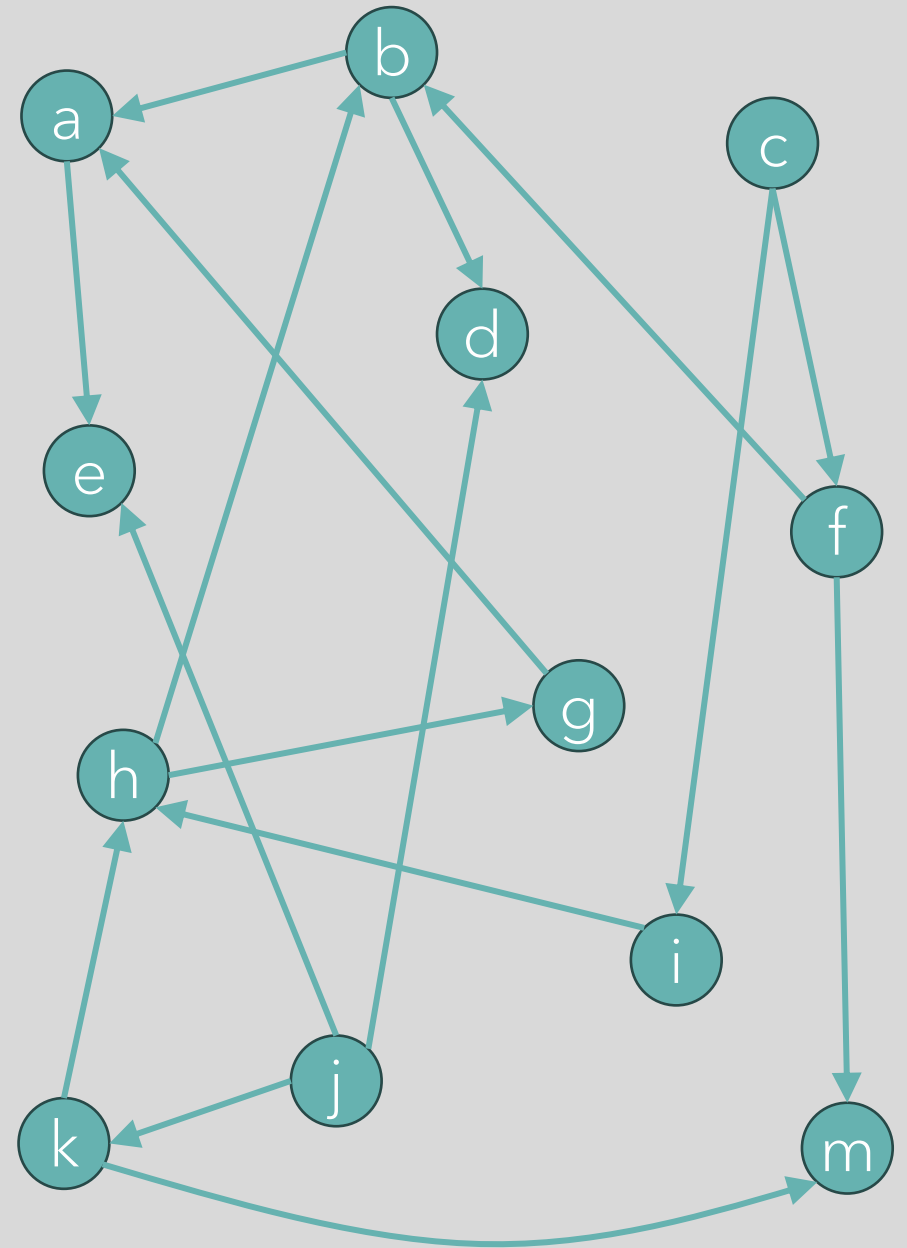
Since all edges are explored at this point, all that remains is to pop each item off the stack, assigning an "out" time as we go.



TOPOLOGICAL SORT

Reminder: A topological sort is any ordering of the vertices such that if there is an edge from x to y , then y comes after x in the ordering.

It only works on directed acyclic graphs.

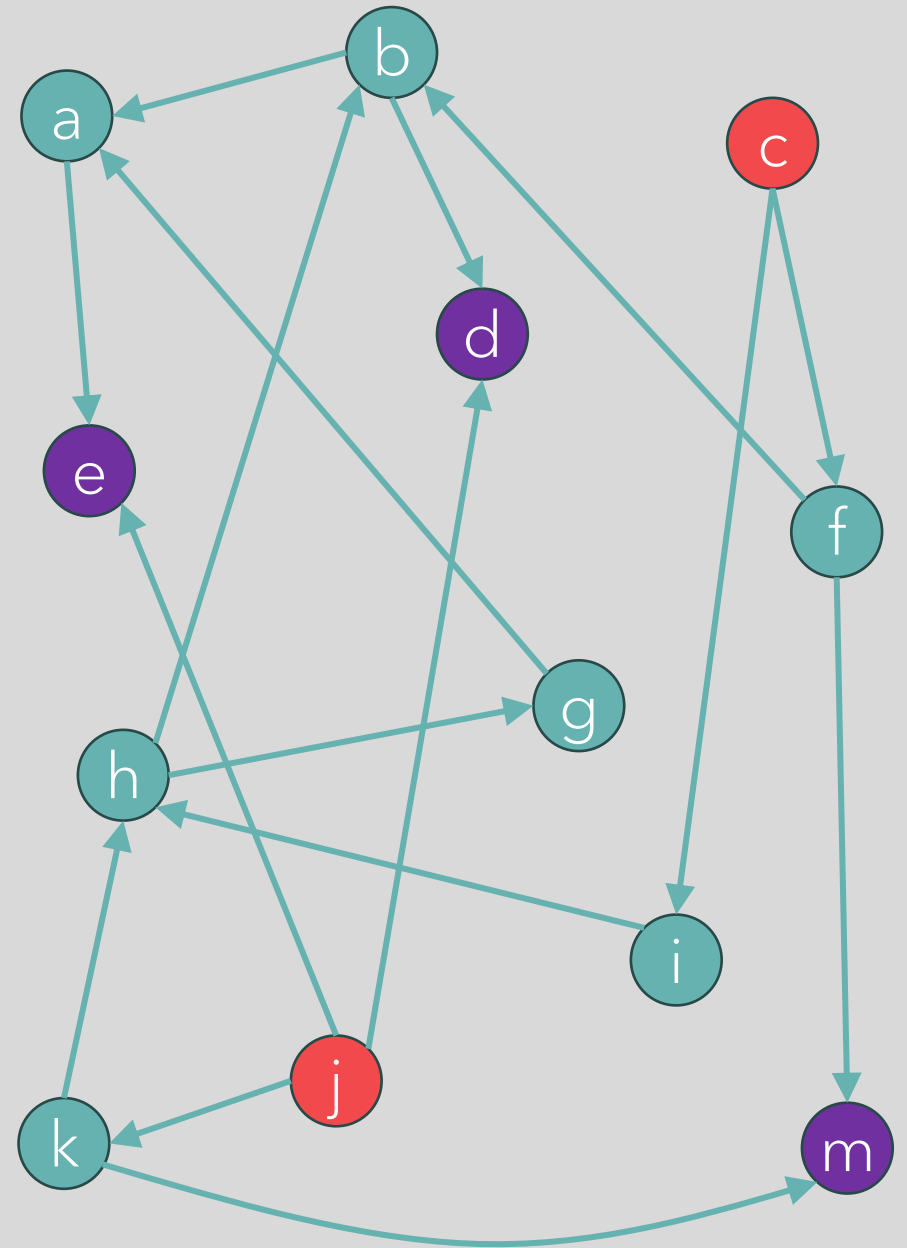


TOPOLOGICAL SORT

One way of addressing topo sort involves looking at *sources* and *sinks*.

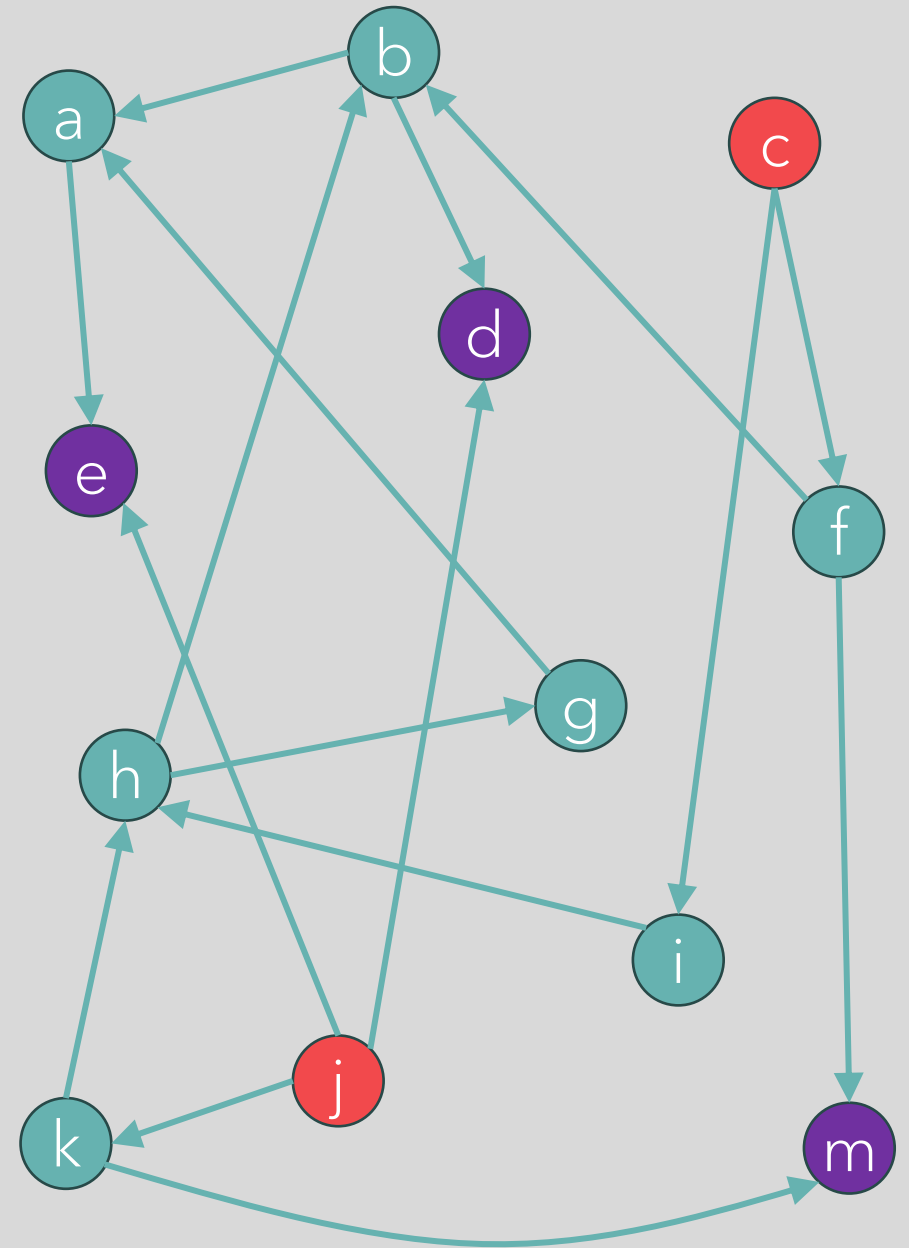
A **source** is a node with no incoming edges.

A **sink** is a node with no outgoing edges.



TOPOLOGICAL SORT

Sources and sinks are easy to find. To find a sink, just follow edges until you hit a dead-end. For a source, follow the edges backwards.

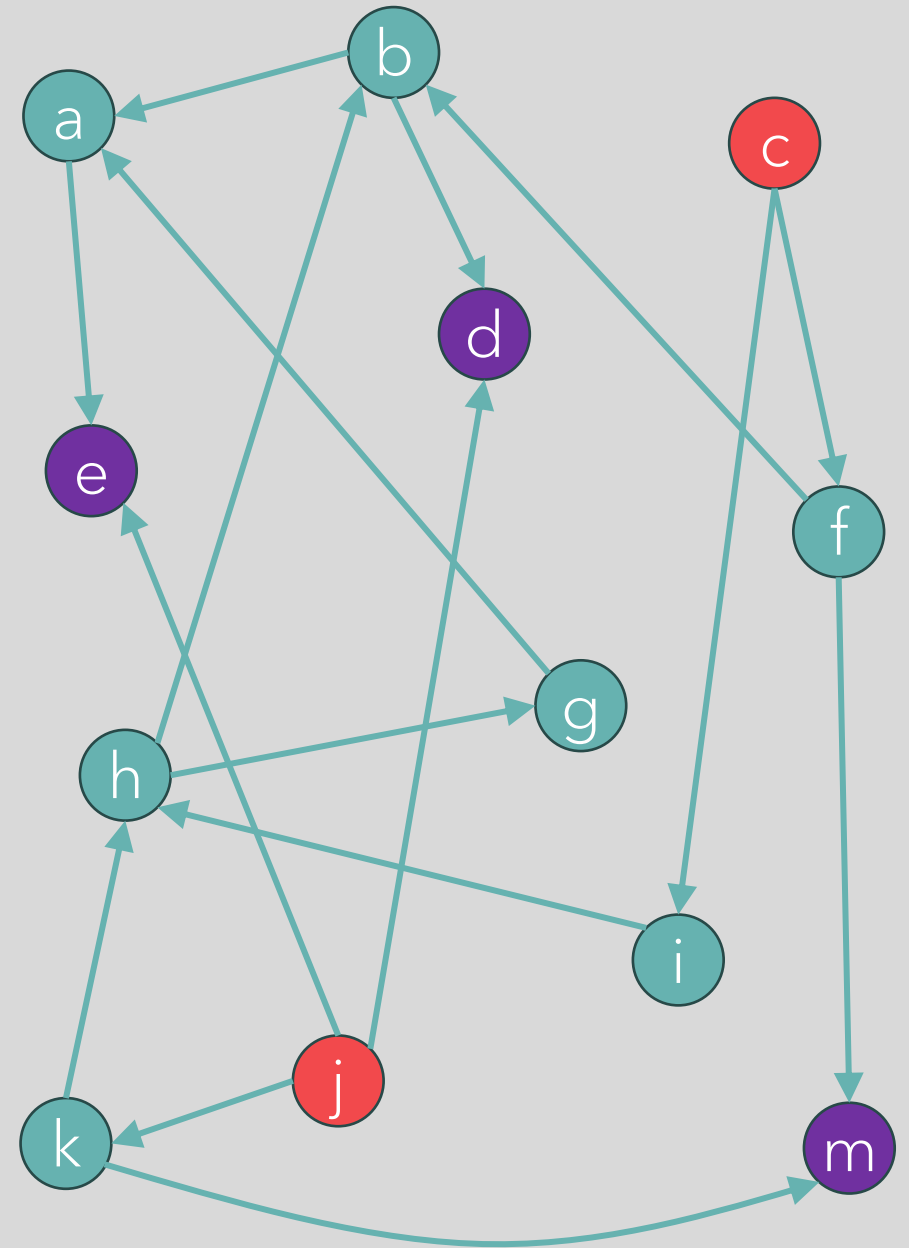


TOPOLOGICAL SORT

A brute-force-ish way of performing topological sort is pretty simple: Find a sink. Make it the last item in your sort. (Why?)

Then remove that node and its edges from the graph and repeat the process.

What's the runtime of this procedure?



TOPOLOGICAL SORT

As you saw in lab, there is a faster approach to topo sort involving DFS, which gives a runtime of $\Theta(V + E)$.

